

# 프로젝트 공부 자료: DexYCB → Revo2 리타게팅 → Floating RL → RB3 실행

작성·코드 확인 기준: 2026-09-11. 이 문서는 현재 저장소를 처음 보는 사람을 위한 한국어 공부 자료다. 구조 → 데이터 → 기구학 → 리타게팅 → RL → 팔 제어 → 검증 순서로 설명하고, 뒤쪽에 원본 비교·수식 유도·코드 읽기·확인 문제를 붙였다. 소스/config, 현재 10,000-update 학습 run의 저장된 설정, 기본 reference의 메타데이터를 대조했다. 과거 실험 결과는 해당 보고서의 결과이며, 이 문서를 작성하면서 학습이나 물리 평가를 다시 실행한 것은 아니다.

여기서 “기본값”은 별도 override 없이 현재 root launcher를 실행할 때의 값이다. Python 클래스 자체의 기본값, 과거 실험의 값, 특정 checkpoint를 만들 때의 값은 서로 다를 수 있다. 재현 시에는 checkpoint뿐 아니라 reference, controller, material, 초기 상태도 고정해야 한다.

수식은 Markdown 미리보기용 LaTeX로 작성했다. VS Code에서 이 파일을 열고 **Ctrl+Shift+V**를 누르면 표와 수식을 함께 볼 수 있다. 수식 표시를 꺼둔 경우 `markdown.math.enabled` 설정을 켜다. 실행 명령·Python 예제·흐름도는 코드 블록으로 유지한다.

생성된 읽기용 사본: 오프라인 HTML · 인쇄 · 공유용 PDF. HTML에는 수식·글꼴이 포함되어 파일 하나로 본문을 공유할 수 있다. 소스·다른 문서·영상 링크는 별도 저장소/파일이 필요하다. Markdown이 편집 원본이며 HTML/PDF는 작성 시점의 사본이다.

## 어떻게 공부하면 좋은가

| 목적           | 권장 순서                    | 읽고 나면 설명할 수 있어야 하는 것              |
|--------------|--------------------------|-----------------------------------|
| 전체 개념, 약 20분 | 1 → 18 → 21              | “사람 동작을 그대로 재생”과 “물리 정책 실행”의 차이   |
| 기구학·리타게팅     | 3-6 → 19.1-19.3 → 20.1   | 좌표계, mimic, 21점 FK, Laplacian 최적화 |
| RL 학습 원리     | 7-11 → 19.4 → 20.2       | 관측·residual·보상·RSI·PPO가 연결되는 방식   |
| 팔 제어·실패 원인   | 12-15 → 19.5-19.7 → 20.3 | 목표와 실제 상태, 추종오차와 특이점, 비교 실험의 한계   |
| 실제 실행·수정 준비  | 16-17 → 20 → 관련 소스/테스트   | 입력 파일, 기본 경로, 변경하면 안 되는 계약        |

이 자료는 전체 소스를 줄마다 번역한 목록이 아니다. 현재 지원 경로의 구현과 중요한 실험을 연결한 학습 지도다. 모든 과거 실험을 현재 기본 설정으로 합치지 않으며, 코드 확인 / 저장된 실험 결과 / 이해를 위한 수식 / 미검증 가설을 구분한다. 처음 읽을 때 모든 링크를 열 필요는 없다.

---

## 목차

1. 전체 구조와 현재 기본 경로
2. 코드 위치와 실행 환경
3. 데이터 전처리와 오른손 변환
4. Revo2 FK와 키포인트
5. REGRIND 리타게팅
6. World 좌표계와 reference 생성
7. Floating-hand 물리환경
8. 정책 observation·action 계약
9. Reward와 성공·실패 판정
10. RSI·랜덤화·curriculum
11. PPO 학습 하이퍼파라미터
12. RB3에 정책을 연결하는 실제 실행
13. 게인·접촉 설정과 floating/arm의 차이
14. Arm fine-tuning
15. 특이점·책상 작업영역 실험
16. 대표 실행 명령
17. 출력 파일·검증·남은 한계
18. 원본 논문·공개 코드와 무엇이 다른가
19. 핵심 수식을 처음부터 연결하기
20. 실제 구현 순서와 코드 읽기
21. 용어·확인 문제·다음 공부

# 1. 전체 구조와 현재 기본 경로

한 문장으로 설명하면, 사람의 손·물체 동작을 로봇손 reference로 바꾸고, floating hand에서 물리적인 파지를 RL로 보정한 뒤, 같은 정책의 손목 목표를 IK로 RB3 관절 목표에 변환해 실행하는 프로젝트다.



중요한 구분:

- 리타게팅은 기구학적 최적화다. 모양을 잘 맞춰도 물리적으로 캔을 잡는다는 보장은 없다.
- Floating RL은 팔 없이 손과 캔의 물리 응답을 학습한다. 팔의 6개 관절각을 출력하는 정책이 아니다.
- Mounted 실행은 실제 mounted 손·캔을 관측하는 closed loop다. 별도 floating simulation의 상태나 미리 저장한 action을 대신 넣지 않는다.
- Offline replay는 저장된 동작을 재생한다. 실제 상태에 따라 정책이 새 action을 내는 online 평가와 다르다.

## 현재 선택되는 것

| 구분                     | 현재 값 / 상태                                                      |
|------------------------|----------------------------------------------------------------|
| 주 사용 sequence          | 20200709_143747_left                                           |
| RL reference           | 해당 sequence의 rb3_revo2_reference_stable.h5 가 있으면 우선 선택         |
| 일반 pipeline 출력         | rb3_revo2_reference.h5 ; stable 파일을 자동 갱신하지 않음                 |
| 기본 floating checkpoint | 2026-09-08_01-28-29_floating_stable_ground_10000/model_9999.pt |
| rl.sh play-arm         | 승인된 video controller + 마지막 마디 compliant contact, 1 env         |
| 이전 arm 경로              | --arm-controller baseline 으로 명시해서 사용                           |

| 구분                        | 현재 값 / 상태                                 |
|---------------------------|-------------------------------------------|
| Arm fine-tuned checkpoint | 별도 선택. 기본 floating checkpoint를 자동 대체하지 않음 |
| 속도·가속도 제한 IK 실험           | opt-in. 현재 video 기본값과 구분                  |
| 실제 로봇 통신                  | 구현·검증 대상 아님. 현재는 simulation               |

기준: 공통 launcher 설정, RL launcher, 승인된 arm 설정.

## 2. 코드 위치와 실행 환경

### 파일을 찾는 지도

| 역할                              | 대표 위치                                |
|---------------------------------|--------------------------------------|
| 사용자가 실행할 명령                     | scripts/README.md, scripts/rl.sh     |
| Dataset 전처리 / 일괄 pipeline       | tools/dexycb_batch                   |
| 리타게팅 CLI                        | retarget_hand_object.py              |
| REGRIND Interaction Mesh 최적화    | retargeter.py, 같은 폴더의 drake_utils.py |
| Revo2 독립 FK                     | revo2_kinematics.py                  |
| World 변환                        | transform_trajectory.py              |
| RB3 FK/IK-reference 생성          | tools/rb3_revo2_ik                   |
| Floating 환경 / PPO config        | config/revo2_floating                |
| Mounted 환경 / transfer config    | config/rb3_revo2                     |
| Observation/action/reward/reset | dexterous/mdp                        |
| 로봇 / 물체 물리 config               | robots, objects/tuna_can.py          |
| 팔 진단·결과 비교                      | tools/arm_diagnostics                |
| 회귀 검증                           | tests, scripts/run_tests.sh          |

regrind/ 는 이 저장소 안에 포함된 upstream-derived 코드이며 별도 Git submodule이 아니다. 원본 LEAP/WUJI의 학습 구조와 PPO 구현을 재사용하고, Revo2/물체/arm 실행 부분을 대응시켰다. 모든 숫자가 원본 embodiment와 같다는 의미는 아니다.

### 로컬에서 확인한 패키지 버전

확인한 Python은 /home/wanjunkim/IsaacLab/.venv/bin/python 이다. 아래는 설치 distribution metadata이며 다른 PC에 그대로 설치돼 있다고 가정하면 안 된다.

| 패키지                    | 버전                       |
|------------------------|--------------------------|
| Isaac Sim              | 6.0.1.0                  |
| isaacsim               | 13.3.0                   |
| isaacsim_physx         | 3.1.1                    |
| isaacsim_rl            | 0.9.0                    |
| RSL-RL ( rsl-rl-lib )  | 5.4.1                    |
| PyTorch                | 2.11.0+cu128             |
| NumPy / SciPy          | 2.5.1 / 1.17.0           |
| Drake / trimesh / h5py | 1.56.0 / 4.11.1 / 3.16.0 |

다른 PC에서는 `ISAAC_SIM_PYTHON` 을 올바른 실행 환경으로 지정한다. Checkpoint, raw dataset, 대형 USD payload/mesh, 로컬 진단 로그는 Git clone만으로 모두 확보된다고 가정하지 않는다.

### 3. 데이터 전처리와 오른손 변환

#### 3.1 입력 선택

preprocess\_dataset.py는 dataset/<sequence>/meta.yaml 의 serial 목록 중 **두 번째 카메라**를 기본으로 고른다. 파일명 정렬상의 두 번째 디렉터리라는 뜻이 아니다. 현재 Isaac reference 준비 코드에는 camera serial 839512060362 검증도 있으므로 임의의 새 dataset을 무조건 처리하는 범용 파이프라인은 아니다.

각 label의 사람 손 joint\_3d , object pose와 ycb\_grasp\_ind 를 읽고, 유효한 finite annotation과 회전행렬을 확인한다. 이 프로젝트의 object asset은 007\_tuna\_fish\_can 이다. 새로운 물체에 자동 대응하는 object recognition 단계는 없다.

전처리 output에는 다음이 들어간다.

| 필드                                        | 의미                                   |
|-------------------------------------------|--------------------------------------|
| mano_joint_coords_original                | 변환 전 사람 손 점                          |
| mano_joint_coords_right_mano21            | 오른손으로 맞춘 sequential MANO21           |
| mano_joint_coords , human_hand_keypoints  | Revo2 correspondence 순서로 재정렬한 사람 손 점 |
| object_pos , object_quat                  | 카메라 기준 물체 pose                       |
| camera / hand-conversion / index metadata | 어떤 convention 변환을 했는지 기록             |

#### 3.2 왼손 → 오른손

왼손이면 물체 local X에 대해 손 점만 반사한다. 프레임마다 물체 pose를 ( $R_o, \mathbf{t}_o$ )라 하면:

$$\begin{aligned} \mathbf{p}_{\text{object}} &= R_o^T (\mathbf{p}_{\text{camera}} - \mathbf{t}_o), \\ \mathbf{p}_{\text{object, right}} &= \text{diag}(-1, 1, 1) \mathbf{p}_{\text{object}}, \\ \mathbf{p}_{\text{camera, right}} &= R_o \mathbf{p}_{\text{object, right}} + \mathbf{t}_o. \end{aligned}$$

오른손 데이터는 이 반사를 하지 않는다. 물체 pose-mesh 자체는 좌우 반사하지 않는다. 이는 점 좌표의 기하학적 변환이며 **오른손 MANO pose/shape parameter를 새로 fitting한 결과는 아니다**. 비대칭 물체라면 반사 후 접촉 면과 파지 가능성이 달라질 수 있다.

#### 3.3 21점 순서

Raw sequential MANO21은 wrist 0, thumb 1-4, index 5-8, middle 9-12, ring 13-16, little 17-20이다. REGRIND/Revo2 쪽에는 다음 permutation을 사용한다.

(0, 5, 6, 7, 9, 10, 11, 17, 18, 19, 13, 14, 15, 1, 2, 3, 4, 8, 12, 16, 20)

| Revo2 semantic index | 의미                   |
|----------------------|----------------------|
| 0                    | wrist semantic point |

| Revo2 semantic index | 의미                                     |
|----------------------|----------------------------------------|
| 1, 2, 3              | index MCP, PIP, DIP                    |
| 4, 5, 6              | middle MCP, PIP, DIP                   |
| 7, 8, 9              | little MCP, PIP, DIP                   |
| 10, 11, 12           | ring MCP, PIP, DIP                     |
| 13, 14, 15           | thumb MCP, PIP, DIP라는 JSON label       |
| 16, 17, 18, 19, 20   | thumb, index, middle, ring, little tip |

그래프 연결선을 만들 때 두 순서를 혼용하면 손 모양이 깨진다. 21이라는 shape만으로 topology를 추측하지 않는다.

### 3.4 Frame 삭제와 재생성 주의

run\_all.sh는 전처리 후 주 sequence의 **앞 12개 전처리 frame**을 제거하는 설정을 갖고 있다. REGRIND\_TRIM\_SEQUENCE , REGRIND\_TRIM\_LEADING\_FRAMES 로 바꿀 수 있다. 과거 다른 단계에서 수행한 앞/뒤 trimming과 stable reference의 frame 수를 이 숫자 하나로 설명할 수는 없다.

Root pipeline의 sequence 필터는 뒤의 retarget/reference 단계에 전달되지만 첫 전처리와 최초 gallery는 dataset 디렉터리들을 열거한다. 학습 중인 입력을 보존하려면 pipeline을 무심코 재실행하지 않는다. dataset/ 는 원본 보존 대상이다.



## 4. Revo2 FK와 키포인트

### 4.1 실제 독립 관절은 6개

다음 순서는 retargeting·reference·policy·actuator mapping에서 공통이다. 각도 단위는 rad이다.

| 순서 | leader joint                 | lower | upper |
|----|------------------------------|-------|-------|
| 0  | right_thumb_metacarpal_joint | 0     | 1.57  |
| 1  | right_thumb_proximal_joint   | 0     | 1.03  |
| 2  | right_index_proximal_joint   | 0     | 1.41  |
| 3  | right_middle_proximal_joint  | 0     | 1.41  |
| 4  | right_ring_proximal_joint    | 0     | 1.41  |
| 5  | right_pinky_proximal_joint   | 0     | 1.41  |

손가락 끝쪽 5개 관절은 follower이다. Thumb distal은 thumb proximal의  $1.0\times$ , 나머지 distal은 각 proximal의  $1.155\times$ , offset은 모두 0이다. 모델은 11개의 움직이는 관절을 포함해도 독립 입력은 6개다.

기구학에서는 mimic를 정확한 종속 관계로 계산한다. 물리 실행에서는 leader로부터 follower position target을 만들고 각 joint limit으로 제한한다. **실제 follower 각도까지 매 physics step에 수학적으로 강제 일치시키는 것은 아니다.** 실제 오차는 따로 측정해야 한다.

### 4.2 21점 계산

각 point의 `parent_link`, `xyz` 를 JSON에서 읽는다.

$$\begin{aligned}T_{\text{base},\ell_i}(\mathbf{q}_{\text{hand}}) &= \text{FK}_{\text{Revo2},\ell_i}(\mathbf{q}_{\text{hand}}, \text{mimic}), \\ \mathbf{p}_{\text{base},i} &= R_{\text{base},\ell_i}(\mathbf{q}_{\text{hand}}) \mathbf{p}_{\text{local},i} + \mathbf{t}_{\text{base},\ell_i}(\mathbf{q}_{\text{hand}}), \\ \mathbf{p}_{\text{world},i} &= R_{\text{world,wrist}} \mathbf{p}_{\text{base},i} + \mathbf{t}_{\text{world,wrist}}.\end{aligned}$$

여기서  $\ell_i$ 는 point  $i$ 의 `parent_link`,  $\mathbf{p}_{\text{local},i}$ 는 JSON의 `xyz` 다.

`Revo2Kinematics.get_keypoints(q)` 는  $(6,) \rightarrow (21,3)$  을 제공한다. NumPy 입력은 NumPy, 지원되는 Torch 입력은 같은 device/dtype의 미분 가능한 Torch 결과를 반환한다. 기본 출력은 hand base 기준이며 world pose는 별도로 적용한다.

Standalone FK는 USD에서 가져온 zero-link transforms와 physics joint/mimic 정보를 사용한다. Retargeter는 packaged URDF를 Drake로 읽고 semantic link offset을 적용한다. 두 경로의 일치는 회귀 검증 대상이다. **21개의 XYZ를 서로 독립인 최적화 변수로 두지 않는다.**

Source of truth는 packaged keypoint JSON과 standalone FK용 JSON이다. 현재 두 복사본을 수동 동기화해야 하는 구조이므로 수정 시 둘의 일치를 확인한다. 좌표 숫자 전체는 이 문서에 다시 복제하지 않는다.

주의: `kp_00_wrist` 는 `right_hand_base_link` 의 원점과 다르다. 현재 local offset은 약  $(0.0156502, 0, 0.00981253)$  m 다. 정책/IK의 wrist frame과 시각화 semantic point 0을 그대로 바꿔 쓰면 안 된다.

구현 기준: revo2\_constants.py, standalone FK, 물리 follower mapping.

## 5. REGRIND 리타게팅

### 5.1 무엇을 최적화하는가

한 frame에서 입력은 사람 손 21점, 물체 pose, 물체 local surface points 50개다. 최적화 대상은 **floating wrist pose + Revo2 6 independent joints**다. 물체 pose는 데모에 고정하고, 로봇손 point는 매 후보 관절각의 FK 결과로 구한다.

실제 pose 저장은 translation 3 + quaternion 4이며 물리적으로는 SE(3)의 6 DoF다. Drake 내부에 11개 손 관절이 있어도 mimic 관계를 적용·투영하여 독립 관절만 풀도록 한다.

### 5.2 Interaction Mesh / Laplacian

1. 사람 손 점과 물체 점을 합쳐 tetrahedral interaction mesh의 adjacency를 구성한다.
2. 각 점이 이웃 점들의 가중 중심에서 얼마나 떨어져 있는지, 즉 Laplacian coordinate를 구한다.
3. 로봇손 FK 점 + 동일 물체 점이 사람 mesh의 관계를 유지하도록 최적화한다.
4. Object frame에서 계산하므로 물체에 대한 손의 상대적인 배치가 핵심이다.

개념식은 다음과 같다. 실제 solver는 Jacobian으로 선형화한 반복 subproblem을 푼다.

$$\begin{aligned} \min_{\mathbf{q}} \quad & \|L_{\text{robot}} V_{\text{robot}}(\mathbf{q}) - \delta_{\text{human}}\|_W^2 + C_{\text{smooth}}(\mathbf{q}, \mathbf{q}_{\text{prev}}) + C_{\text{nominal}}(\mathbf{q}) \\ \text{subject to} \quad & \mathbf{q}_{\min} \leq \mathbf{q} \leq \mathbf{q}_{\max}, \\ & \mathbf{q} \in \mathcal{C}_{\text{mimic}} \cap \mathcal{C}_{\text{nonpenetration}}, \\ & \|\Delta \mathbf{q}\|_2 \leq s_{\text{step}}. \end{aligned}$$

$C_{\text{smooth}}$ 는 frame 간 smoothness 비용이고,  $C_{\text{nominal}}$ 은 nominal trajectory가 있을 때만 쓰는 비용이다.  $\mathcal{C}_{\text{nonpenetration}}$ 은 각 반복에서 선형화하는 비침투 제약이며,  $s_{\text{step}}$ 은 `step_size`다. 위 식은 개념식으로, 실제로는 configuration increment  $\Delta \mathbf{q}$ 에 대한 subproblem을 푼다.

구현상 주의: 첫 pose의 `init_t=True`에서는 cone step-size 제약을 제거하고 다시 푼다. 또한 후처리 backtracking은 `max_iters=0`으로 비활성화돼 있다. 두 동작 모두 확인한 upstream에도 존재한다. 따라서 “모든 반복에 0.2 제한이 적용된다”거나 “비선형 penetration이 마지막에 반드시 제거된다”고 해석하면 안 된다. 정확한 변수 차원과 선형화는 19.3절에서 설명한다.

단순히 사람 21점과 로봇 21점의 world-space MSE만 최소화하는 방식은 아니다. 또한 collision 제약이 있어도 마찰·힘·동적 파지를 최적화하는 physics solver는 아니다.

### 5.3 실제 batch 경로의 설정

| 항목                         | 값 / 주의                              |
|----------------------------|-------------------------------------|
| Subproblem solver          | <b>Clarabel</b> ; batch wrapper가 명시 |
| 단독 retarget CLI의 기본 solver | Mosek; 라이선스 필요할 수 있음. Batch와 다름     |
| Laplacian weight           | 10                                  |
| Hand point별 추가 배율          | Revo2 경로에서는 기본 1                    |

| 항목                            | 값 / 주의                                                                                       |
|-------------------------------|----------------------------------------------------------------------------------------------|
| Temporal smooth weight        | 0.5                                                                                          |
| Step-size cone bound          | 0.2; 혼합 configuration increment의 norm, 0.2 m라고 해석하지 않음                                       |
| Collision detection threshold | 0.1 m                                                                                        |
| Non-penetration tolerance     | batch는 0.002 m; 단독 Revo2 CLI 기본은 0.001 m                                                     |
| Iterations                    | 처음 성공 pose를 얻을 때 최대 50, 이후 frame당 최대 10                                                      |
| Cost 조기 수렴                    | <code>np.isclose(cost, last_cost, atol=1e-10)</code> ; NumPy 기본 relative tolerance도 적용       |
| Nominal tracking              | 초기 weight 5, decay $\exp(-f/10)$ ; $f$ 는 frame index, nominal trajectory가 제공될 때의 항목          |
| Wrist initialization          | Revo2 DexYCB에서 semantic point rigid alignment ( <code>auto</code> → <code>keypoints</code> ) |
| 초기 hand joint                 | 6개 모두 0                                                                                      |
| 추가 초기 wrist rotation          | 기본 <code>(0,0,0)</code> deg; 옛 180° 실험을 항상 적용하지 않음                                           |
| Source fps / interpolation    | 30 Hz / DexYCB 기본 factor 1                                                                   |
| Table                         | packaged Revo2 retarget object config의 <code>table_height=None</code> ; arm workcell 검사와 별개  |

Frame은 순차적으로 처리하고 마지막 **성공한** configuration을 다음 초기값으로 쓴다. 실패 frame은 NaN/failure index와 solver 정보를 남긴다. 이전 frame을 복사해 성공처럼 저장하지 않는다.

기준: batch 호출, CLI, 최적화 구현.

## 6. World 좌표계와 reference 생성

### 6.1 Camera → tabletop world

현재 batch는 `--camera-frame-convention dexycb_y_down` 을 명시한다. Camera `+X` 는 영상 오른쪽, `+Y` 는 아래, `+Z` 는 전방이다.

$$R_0 = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}, \quad R_{\text{world,camera}} = R_z(\psi_{\text{sequence}}) R_0.$$

따라서 **camera -Y가 world +Z**가 된다. 물체의 local Z가 뒤집혀 있다는 이유로 중력축까지 뒤집지 않는다.

첫 frame mesh vertices를 변환한 상대 높이의 최솟값을 사용한다.

$$\begin{aligned} R_{\text{world,object0}} &= R_{\text{world,camera}} R_{\text{camera,object0}}, \\ \mathbf{p}_{\text{object,desired}} &= \begin{bmatrix} 0.4 \\ 0.0 \\ -\min_i (R_{\text{world,object0}} \mathbf{v}_i)_z \end{bmatrix}, \\ \mathbf{t}_{\text{world,camera}} &= \mathbf{p}_{\text{object,desired}} - R_{\text{world,camera}} \mathbf{p}_{\text{camera,object0}}. \end{aligned}$$

$\mathbf{v}_i$ 는 object-local mesh vertex이고, 위치 단위는 m이다. 같은 rigid transform을 전체 frame에 적용한다.

$$\begin{aligned} T_{\text{world,object}}[t] &= T_{\text{world,camera}} T_{\text{camera,object}}[t], \\ T_{\text{world,wrist}}[t] &= T_{\text{world,camera}} T_{\text{camera,wrist}}[t], \\ \mathbf{p}_{\text{world,MANO},i}[t] &= R_{\text{world,camera}} \mathbf{p}_{\text{camera,MANO},i}[t] + \mathbf{t}_{\text{world,camera}}. \end{aligned}$$

이 방식은 첫 mesh의 가장 낮은 점을  $Z=0$ 에 둔다. 데모에서 물체가 기울어져 있었다면 그것만으로 바닥면 전체가 수평이 되는 것은 아니다. 별도 stable reference의 leveling 이력과 구분해야 한다.

단독 transform 도구에는 `object_upright` 모드도 있다. 그것은 원하는 첫 object pose로 다음 transform을 구하며 local +Z가 world +Z인 upright 회전을 요구한다. 현재 batch의 camera-gravity 모드와 혼동하지 않는다.

$$T_{\text{world,camera}} = T_{\text{world,object,desired}} T_{\text{camera,object0}}^{-1}.$$

| Sequence              | Batch world yaw |
|-----------------------|-----------------|
| 20200709_143626_right | +120°           |
| 20200709_143703_right | -120°           |
| 20200709_143747_left  | +150°           |
| 20200709_143826_left  | -30°            |
| 20200709_143907_right | -120°           |

새 sequence에는 ISAAC\_ALIGNMENT를 먼저 검토해야 한다.

6.2 Quaternion·관절 순서 계약

| 단계 / API                                     | Quaternion 순서     |
|----------------------------------------------|-------------------|
| 전처리 NPZ                                      | wxyz              |
| Retargeted H5                                | xyzw              |
| World trajectory                             | wxyz              |
| 최종 RB3+Revo2 reference / floating rollout    | xyzw              |
| 현재 설치된 프로젝트 Isaac Lab 실행 경로 / SciPy Rotation | xyzw              |
| USD Gf quaternion의 real + imaginary 표현       | wxyz 로 명시 변환해서 취급 |

항상 파일의 quat\_convention 을 읽는다. “Isaac은 항상 wxyz” 같은 다른 버전의 관행을 현재 코드에 그대로 적용하면 안 된다.

최종 joint 순서는 [base, shoulder, elbow, wrist1, wrist2, wrist3, Revo2 6 leaders] 다. 이름으로 대응시키며 articulation 내부 배열 순서를 추측하지 않는다.

6.3 Offline RB3 IK

RB3730Kinematics는 USD에서 검증한 rb3\_model.json의 serial chain과 mount transform을 사용한다. 직접 USD를 매 IK 호출마다 다시 로드하지 않는다.

$$\mathbf{r}(\mathbf{q}) = \begin{bmatrix} 10(\mathbf{p}_{\text{FK}}(\mathbf{q}) - \mathbf{p}_{\text{target}}) \\ \text{rotvec}\left(R_{\text{target}}^T R_{\text{FK}}(\mathbf{q})\right) \end{bmatrix}.$$

| 항목                       | Standalone 기본값                                                   |
|--------------------------|------------------------------------------------------------------|
| Solver                   | SciPy least_squares , bounded trf                                |
| Position acceptance      | $10^{-4} \text{ m} = 0.1 \text{ mm}$                             |
| Orientation acceptance   | $10^{-3} \text{ rad} \approx 0.0573^\circ$                       |
| Position residual weight | 10                                                               |
| ftol , xtol , gtol       | 각각 $10^{-12}$                                                    |
| max_nfev                 | 800; 현재 mounted 호출은 300을 전달                                      |
| 초기 후보                    | warm start, neutral, elbow/base alternative; 중복 제거               |
| 여러 성공 후보                 | 이전 q에 가장 가까운 후보 선택                                               |
| Joint position limits    | elbow $\pm 150^\circ$ , 나머지 $\pm 360^\circ$ ; 현재 simulation 모델 값 |
| 기본 Jacobian              | finite difference; warm-first shortcut에는 analytic Jacobian 사용    |

실패 시 optimizer success와 strict pose success를 따로 기록한다. 성공 판정은 FK 오차·finite·joint bounds까지 확인한다. Offline strict pose success는 원래 속도의 actuator 추종이나 collision-free grasp 성공을 보장하지 않는다.

6.4 현재 stable reference의 실제 내용

파일: outputs/isaac/dexycb/20200709\_143747\_left/rb3\_revo2\_reference\_stable.h5

직접 읽어 확인한 값:

| 항목                            | 값                                                                       |
|-------------------------------|-------------------------------------------------------------------------|
| Frames / fps                  | 38 / 30                                                                 |
| 샘플 첫 시각 → 마지막 시각 간격           | $\frac{38-1}{30}$ s $\approx$ 1.23333 s; 영상 hold나 runtime wall time과 다름 |
| ik_success 실패 index           | 없음                                                                      |
| Quaternion                    | xyzw                                                                    |
| Object leveling               | true, clearance 0, table height 0                                       |
| 저장된 초기 mesh min Z             | 약 $-3.47e-18$ m                                                         |
| Object 초기 위치                  | [0.400000, 0.000000, 0.012636] m                                        |
| Wrist base 초기 위치              | [0.291512, -0.065391, 0.144138] m                                       |
| Wrist-object origin           | [-0.108488, -0.065391, +0.131502] m ; 거리 약 0.1826 m                     |
| 저장된 추가 local wrist correction | identity / [0,0,0] deg                                                  |
| 현재 phase 분모                   | loader fallback상 38-1; config offset 0                                  |

18.26 cm는 캔 origin과 hand base origin의 거리다. 손끝과 캔 표면 사이 여유 거리가 아니다. Object quaternion이 identity가 아니어도 mesh local 축 convention 때문에 캔이 바르게 놓일 수 있다.

이 파일의 source\_retargeting\_file 은 과거 outputs/floating/random\_can\_replay/20200709\_143747\_left\_random\_rollout.h5 를 가리키고, source\_start\_frame=2 , floating alignment/leveling metadata가 있다. 따라서 현재 stable 파일을 방금 일반 pipeline이 만든 순수 retargeting 결과라고 부르면 안 된다. 전체 과거 생성 과정을 메타데이터만으로 완전히 재구성할 수는 없다.

기본 RL loader는 이 파일의 wrist/hand/object reference를 그대로 쓴다. zero 의 정확한 뜻은 선택한 reference에 새 residual 을 더하지 않는 것이며, 자동으로 원본 human demo만 재생한다는 뜻이 아니다.

## 7. Floating-hand 물리환경

환경은 Revo2 articulation, dynamic tuna can, 고정 table, 조명으로 구성된다. RB3는 존재하지 않는다. 손목은 외력/토크 기반 Cartesian impedance로 움직이고 손가락은 implicit joint drive로 움직인다.

### 공통 시간·물리 설정

| 항목                                                | Floating task 값                                                                |
|---------------------------------------------------|--------------------------------------------------------------------------------|
| Physics dt                                        | $\Delta t_{\text{physics}} = \frac{1}{120} \text{ s} \approx 8.333 \text{ ms}$ |
| Decimation                                        | 4                                                                              |
| Policy / reference update                         | 30 Hz = 33.333 ms                                                              |
| Episode length upper bound                        | 10 s; 실제로는 reference 종료/실패가 먼저 발생 가능                                           |
| Full training env 수 / spacing                     | 4096 / 0.75 m                                                                  |
| Smoke env 수 / spacing                             | 16 / 1.0 m                                                                     |
| Play env 수 / spacing                              | 1 / 1.5 m                                                                      |
| PhysX solver type                                 | 1 (TGS)                                                                        |
| Scene maximum position / velocity iterations      | 64 / 4                                                                         |
| Robot articulation position / velocity iterations | 32 / 2                                                                         |
| Tuna position / velocity iterations               | 16 / 2                                                                         |
| Bounce velocity threshold                         | 0.2 m/s                                                                        |
| Friction offset / correlation distance            | 0.01 / 0.00625 m                                                               |
| GPU rigid contacts / patches capacity             | 2**20 / 2**19                                                                  |
| Robot self collisions                             | OFF                                                                            |
| Floating robot gravity                            | OFF ( disable_gravity=True )                                                   |
| Object gravity                                    | ON; scene gravity curriculum 또는 eval full gravity 적용                           |

손의 중력이 꺼져 있다는 것은 physics가 꺼져 있다는 뜻은 아니다. 접촉과 관성은 존재한다. 반면 assembled arm+hand는 gravity가 켜져 있다. 두 embodiment의 중요한 물리적 차이다.

### Floating wrist impedance

$$\begin{aligned}\mathbf{F} &= K_{p,\text{pos}} (\mathbf{p}_{\text{target}} - \mathbf{p}_{\text{actual}}) - K_{d,\text{pos}} \mathbf{v}_{\text{actual}}, \\ \boldsymbol{\tau} &= K_{p,\text{rot}} \text{rotvec} (Q_{\text{target}} \otimes Q_{\text{actual}}^{-1}) - K_{d,\text{rot}} \boldsymbol{\omega}_{\text{actual}}.\end{aligned}$$

수식에서  $Q$ 는 단위 quaternion,  $\mathbf{q}$ 는 joint vector,  $\otimes$ 는 quaternion 곱이다. rotvec는 최단 회전의 axis-angle vector를 뜻한다.



| 계수     | 값   | 단위        |
|--------|-----|-----------|
| kp_pos | 300 | N/m       |
| kd_pos | 30  | N·s/m     |
| kp_rot | 3   | N·m/rad   |
| kd_rot | 0.3 | N·m·s/rad |

SE3ImpedanceActionTerm은 root에 힘을 적용하고, 작은 dummy root 관성의 문제를 줄이기 위해 토크를 articulation body 질량 비율로 분배한다. Mounted 경로는 pose decoder만 재사용하고 **이 floating wrench를 RB3 전체에 다시 적용하지 않는다.**

Floating table은 1.2×1.2×0.1 m, center (0.4,0,-0.05) 다. 책상·로봇 받침까지 있는 mounted workcell과 geometry가 같지 않다.

## 8. 정책 observation·action 계약

### 8.1 Actor 67차원

아래 slice는 Python 반열린 구간 `[start:end]` 다. 원본 actor 경로와 mounted 경로가 같은 `FloatingObservationsCfg` 를 사용한다.

| Slice                | 내용                                           | 차원 |
|----------------------|----------------------------------------------|----|
| <code>[0:3]</code>   | 현재 object position                           | 3  |
| <code>[3:9]</code>   | 현재 object rotation 6D                        | 6  |
| <code>[9:15]</code>  | Wrist position history, 길이 2                 | 6  |
| <code>[15:27]</code> | wrist rotation 6D history, 길이 2              | 12 |
| <code>[27:39]</code> | hand 6 leader position-default, history 길이 2 | 12 |
| <code>[39:51]</code> | action manager의 last action                  | 12 |
| <code>[51:52]</code> | reference phase                              | 1  |
| <code>[52:61]</code> | action 기준 wrist position + rotation 6D       | 9  |
| <code>[61:67]</code> | action 기준 hand 6 leader joint positions      | 6  |

History의 쌓기·flatten·reset은 기존 ObservationManager에 맡긴다. 별도 adapter에서 순서를 다시 만들지 않는다. Rotation 6D는 `R[..., :2].reshape(...)`, 즉 회전행렬의 앞 두 column을 기존 flatten 순서로 사용한다. 임의의 Euler 6값으로 바꾸면 안 된다.

현재 상태 position은 우선 `world - env_origin` 이다. XY placement augmentation을 켜면 positional observation에서 동일한 episode translation을 한 번 더 뺀다. 이는 위치 이동에 대한 canonicalization이며 **캔 yaw까지 canonicalize하는 경로는 아니다**. 손목 실제 상태 자리에 RB3 6 joint를 넣거나 desired wrist pose를 actual observation처럼 넣지 않는다.

### 8.2 Critic 94차원

Actor와 같은 종류·순서의 67차원 항목을 noise/delay 없이 받고, 다음을 추가한다.

| Slice                | 추가 정보                          | 차원 |
|----------------------|--------------------------------|----|
| <code>[67:70]</code> | object linear velocity         | 3  |
| <code>[70:73]</code> | object angular velocity        | 3  |
| <code>[73:88]</code> | 실제 5개 fingertip body positions | 15 |
| <code>[88:94]</code> | Revo2 leader joint velocity    | 6  |

5 fingertip은 thumb/index/middle/ring/pinky의 `right*_touch_link` 다. Retargeting semantic 21개 전체도 아니고, JSON tip offset 5개와 무조건 같은 좌표도 아니다. Rigid tuna에는 articulated object joint observation을 넣지 않는다.

## 8.3 Action 12차원과 decoder

| Slice  | 의미                             | 30 Hz에서 component당 scale                                                           |
|--------|--------------------------------|------------------------------------------------------------------------------------|
| [0:3]  | Wrist position residual        | $s_p = (1.0 \text{ m/s}) \Delta t_{\text{policy}} \approx 0.0333333 \text{ m}$     |
| [3:6]  | Wrist rotation-vector residual | $s_R = (3.2 \text{ rad/s}) \Delta t_{\text{policy}} \approx 0.1066667 \text{ rad}$ |
| [6:12] | Revo2 leader joint residual    | $s_h = (3.2 \text{ rad/s}) \Delta t_{\text{policy}} \approx 0.1066667 \text{ rad}$ |

$$\begin{aligned}
 \bar{\mathbf{a}} &= \text{clip}(\mathbf{a}_{\text{policy}}, -1, +1), \\
 \mathbf{p}_{\text{target}} &= \mathbf{p}_{\text{reference}}[\text{phase}] + s_p \bar{\mathbf{a}}_{0:3}, \\
 Q_{\text{target}} &= \text{Quat}(s_R \bar{\mathbf{a}}_{3:6}) \otimes Q_{\text{reference}}[\text{phase}], \\
 \mathbf{q}_{\text{hand,target}} &= \text{clip}(\mathbf{q}_{\text{hand,reference}}[\text{phase}] + s_h \bar{\mathbf{a}}_{6:12}, \mathbf{q}_{\text{hand,min}}, \mathbf{q}_{\text{hand,max}}), \\
 q_{\text{follower},j}^{\text{target}} &= \text{clip}\left(m_j q_{\text{leader}(j)}^{\text{target}} + b_j, q_{\text{follower},j}^{\text{min}}, q_{\text{follower},j}^{\text{max}}\right).
 \end{aligned}$$

clip은 componentwise 제한, Quat은 회전벡터를 단위 quaternion으로 바꾸는 연산이다.  $m_j, b_j$ 는 follower  $j$ 의 mimic multiplier와 offset이며, action slice는 Python의 0-based 반열린 구간과 같다.

회전 residual은 quaternion 왼쪽 곱, world-axis rotation-vector convention이다. 표의 scale은 각 component의 크기이므로 3D vector norm 한계와 같지 않다. `motion_target` 이 기준이므로 residual을 전 step target에 계속 누적하는 구조가 아니며 두 번 더하지 않는다.

앞의 6차원은 **RB3 joint residual**이 아니다. 과거 combined task의 `arm=0.05`, `hand=0.15` residual 설정을 현재 floating/online 정책에 적용해서 해석하면 안 된다.

## 8.4 정규화와 평가

Actor-critic은 각각 observation normalization을 사용한다. Actor 마지막 선형층의 mean 출력 weight/bias를 0으로 초기화하므로 초기 deterministic residual mean은 0이다. 학습 action sampling의 Gaussian std가 0이라는 뜻은 아니다.

FrozenPolicyAdapter는 기존 observation을 그대로 frozen actor에 전달한다. `eval()`, inference mode, gradient 비활성화를 사용하고 state\_dict가 변하지 않았는지 검사한다. Normalizer를 두 번 적용하거나 평가 중 통계를 갱신하지 않는다. Observation history와 phase의 owner는 기존 manager다.

기준: Floating config, observations.py, actions.py.

## 9. Reward와 성공·실패 판정

### 9.1 Object-centric 50-point tracking

같은 object-local surface points 50개를 reference pose와 actual pose로 각각 변환한다.

$$\begin{aligned}\mathbf{p}_{\text{ref},i} &= R_{\text{object,ref}} \mathbf{p}_{\text{local},i} + \mathbf{t}_{\text{object,ref}}, \\ \mathbf{p}_{\text{sim},i} &= R_{\text{object,sim}} \mathbf{p}_{\text{local},i} + \mathbf{t}_{\text{object,sim}}, \\ e_{\text{keypoint}} &= \frac{1}{50} \sum_{i=1}^{50} \|\mathbf{p}_{\text{ref},i} - \mathbf{p}_{\text{sim},i}\|_2.\end{aligned}$$

원형 캔이라고 yaw 오차를 없애거나 최소 등가 회전을 찾지는 않는다. **현재 reward는 symmetry-aware가 아니다.**

### 9.2 현재 활성 reward 전체

$\mathbf{a}$ 는 action manager의 12차원 action이며, 손가락 rad나 actuator torque가 아니다.  $\theta$ 는 quaternion 회전각 오차다. 표의  $\Delta \mathbf{p}$ ,  $\Delta \mathbf{v}$ ,  $\Delta \omega$ 는 각각 actual-reference 차이다. Action 크기·변화·범위 초과 항은 다음과 같다.

$$\begin{aligned}E_a &= \frac{1}{12} \sum_{j=1}^{12} a_j^2, \\ E_{\Delta a} &= \frac{1}{12} \sum_{j=1}^{12} (a_j - a_{\text{prev},j})^2, \\ E_{\text{bounds}} &= \sum_{j=1}^{12} \max(\text{abs}(a_j) - 1, 0).\end{aligned}$$

| 항목                      | 원시 reward 식                                                 | Weight |
|-------------------------|-------------------------------------------------------------|--------|
| Object keypoint         | $\exp\left(-\frac{e_{\text{keypoint}}}{0.02}\right)$        | 1.5    |
| Object linear velocity  | $\exp\left(-\frac{\ \Delta \mathbf{v}\ _2^2}{1.0^2}\right)$ | 1.0    |
| Object angular velocity | $\exp\left(-\frac{\ \Delta \omega\ _2^2}{3.14^2}\right)$    | 1.0    |
| Wrist position          | $\exp\left(-\frac{\ \Delta \mathbf{p}\ _2}{0.02}\right)$    | 0.05   |
| Wrist orientation       | $\exp\left(-\frac{\theta}{0.2}\right)$                      | 0.05   |
| Action magnitude        | $\exp\left(-\frac{E_a}{1.0^2}\right)$                       | 0.5    |
| Action rate             | $\exp\left(-\frac{E_{\Delta a}}{0.5^2}\right)$              | 1.0    |
| Action bounds           | $\exp\left(-\frac{E_{\text{bounds}}}{1.0}\right)$           | 1.0    |
| Early termination       | $\mathbf{1}_{\text{premature termination}}$                 | -10.0  |

Magnitude/rate는 음수 penalty가 아니라 **작을수록 큰 양의 exponential reward**다. 모든 tracking 항목을  $\exp(-e^2/\sigma^2)$ 로 통일해 설명하면 틀린다. Keypoint/wrist 위치와 회전은 위 표처럼 선형 error exponent다.

설치된 Isaac Lab RewardManager의 한 step reward는 다음과 같다. 따라서 표의 weight 합과 한 step의 최종 reward는 같지 않으며, action-rate 식 자체는 dt로 나눈 velocity가 아니다.

$$r_{\text{step}} = \Delta t_{\text{policy}} \sum_k w_k r_k^{\text{raw}}.$$

9.3 종료와 실제 파지의 차이

| 조건                      | 현재 값                                                                           |
|-------------------------|--------------------------------------------------------------------------------|
| Reference 종료            | $f_{\text{current}} + 1 \geq T_{\text{demo}}$                                  |
| success term            | 위 reference 종료와 같은 boolean                                                     |
| Object deviation 실패     | $e_{\text{keypoint}} > 0.15 \text{ m}$                                         |
| Hand far from object 실패 | $\ \mathbf{p}_{\text{wrist}} - \mathbf{p}_{\text{object}}\ _2 > 0.5 \text{ m}$ |
| Time limit              | 10 s                                                                           |

success 는 안정 파지를 직접 검출하는 센서 판정이 아니다. 끝 frame에서 다른 failure와 동시에 true가 될 수 있어 비교 분석기는 동시 발생을 별도 기록한다.

기존 arm 비교 분석의 보조 lift/hold proxy는 **마지막 0.2 s에서 초기 대비 높이 증가가 계속 0.1 m 이상이고, can-robot contact >0.01 N인 sample 비율이 80% 이상**인 조건이다. 들어올린 뒤 마지막 높이가 0.05 m 미만이면 drop proxy도 본다. 이것은 기존 RL reward/termination을 바꾼 새 성공 기준이 아니라 결과 해석을 위한 보조 지표다.

기준: rewards.py, reward/termination config, 비교 분석.

## 10. RSI·랜덤화·curriculum

### 10.1 RSI와 reset 순서

RSI는 episode를 항상 첫 frame에서 시작하지 않고 reference frame 중 하나에서 시작하는 방식이다. 현재 floating training은 uniform으로 0 ... T-2 중 선택한다. Play는 RSI OFF, frame 0부터 시작한다.

reset 대상 env 선택

→ reference frame / phase 선택

→ XY placement 선택

→ 그 frame의 leader/mimic positions·velocities 적용

→ floating이면 wrist root pose·velocity 적용

→ object pose·velocity 적용

→ mounted이면 선택된 reference/state 이후 IK reset 동기화

→ history/action/response buffer를 새 episode 상태로 시작

손 joint reset noise는 기본  $\pm 0.02$  rad다. 현재 floating training은 object position/rotation reset noise를 0으로 덮어쓴다. 즉 캔을 무작정 띄우거나 기울이는 perturbation 대신 전체 reference를 XY로 옮긴다. RSI가 동작 중간 frame을 뺏으면 그 frame의 캔이 공중에 있는 것은 정상적인 RSI 의미다.

초기 reset의 state write는 허용된 초기화다. 정상 closed-loop 실행 중 매 step robot/object state를 reference로 덮어써 추종을 위조하지 않는다. 다만 training의 random push는 아래처럼 명시적 velocity perturbation이다.

### 10.2 Floating training 기본 randomization

| 항목                                      | 범위                                        | 시점 / 의미                                     |
|-----------------------------------------|-------------------------------------------|---------------------------------------------|
| Object 시작 X / Y                         | [0.40,0.50] / [-0.20,0.20] m              | Reset; hand/object/reference 공통 translation |
| Object yaw / Z placement                | 추가 randomization 없음                       | XY만 바꿈                                      |
| Revo2 static/dynamic friction           | [0.7,1.3]                                 | Startup, 250 material buckets               |
| Object friction                         | [0.5,1.2]                                 | Startup, 250 buckets                        |
| Table friction                          | [0.6,1.2]                                 | Startup, 250 buckets                        |
| Restitution                             | 0                                         | Material randomization에서도 0                 |
| Revo2 mass multiplier                   | [0.9,1.1]                                 | Startup                                     |
| Revo2 Kp/Kd multiplier                  | [0.8,1.2] 각각                              | Startup, log-uniform                        |
| Object mass multiplier                  | [0.85,1.15]                               | Startup; nominal 0.15 kg → 0.1275-0.1725 kg |
| Object COM X/Y                          | 각각 $\pm 0.002$ m                          | Startup                                     |
| Object COM Z                            | $\pm 0.001$ m                             | Startup                                     |
| Object/wrist position observation noise | componentwise uniform $\pm 0.002$ m       | Actor 관측                                    |
| Object/wrist rotation observation noise | random axis, uniform angle $\pm 0.02$ rad | Actor 관측                                    |

| 항목                           | 범위                                   | 시점 / 의미                       |
|------------------------------|--------------------------------------|-------------------------------|
| Hand joint observation noise | componentwise uniform $\pm 0.02$ rad | Actor 관측                      |
| Observation latency          | 0-2 observation steps                | Reset에서 선택, continuous lag 설정 |

30 Hz에서는 2 observation steps가 약 66.7 ms다. Object pose의 위치/회전 lag는 묵고 wrist 위치/회전도 묵는다. Hand joint는 별도 group이다. 이는 별도 action delay 66.7 ms를 추가했다는 뜻이 아니다. 현재 active config에 독립적인 action-latency randomizer를 더했다고 해석하지 않는다.

`enable_corruption=False` 여도 observation 함수의 `apply_noise=True` 와 delay buffer가 따로 실행되므로 training noise가 꺼졌다는 뜻은 아니다. Critic의 대응 항목은 이 noise/delay를 적용하지 않는다.

RB3-only friction/mass/gain event는 floating config에서 제거한다. 보존된 combined task에는 arm 범위가 별도로 있지만 현재 video transfer는 randomization OFF다.

### 10.3 Gravity curriculum

기준 counter는 `env.common_step_counter` : **vectorized policy/environment step** 수다. Env 수를 곱한 transition 수도, physics substep 수도 아니다. 아래의 *g*는 아래 방향 중력 크기이며 실제 Z 성분은 음수다.

| Counter 시작 | <i>g</i> 범위 [m/s²] |
|------------|--------------------|
| 0          | 0                  |
| 20,000     | 0-1                |
| 30,000     | 0.5-2              |
| 40,000     | 1-3                |
| 50,000     | 2-4                |
| 60,000     | 3-5                |
| 70,000     | 4-6                |
| 80,000     | 5-7                |
| 90,000     | 6-8                |
| 100,000    | 7-9                |
| 110,000    | 8-9.81             |
| 120,000    | 9-9.81             |
| 130,000    | 9.81 고정            |

Stage를 reset event에서 선택한다. 24 rollout steps/update이므로 중단 없는 run에서 full-gravity threshold는 대략 5,417 update 부근이다. Scene gravity randomizer의 적용 단위까지 각 환경의 독립 gravity라고 가정하지 않는다.

### 10.4 Random push curriculum

Robot과 object 각각 1-5 s 간격 event로 `push_by_setting_velocity` 를 호출한다. 설치된 구현은 현재 root velocity에 표의 범위에서 뽑은 **속도 증분을 더한 뒤** simulator에 쓴다. 함수 이름이나 docstring만 보고 아래 값을 최종 절대 속도 목표라고 해석하지 않

는다. Force impulse를 직접 주는 구현은 아니다.

| Counter 시작 | XYZ velocity 증분 각 축 | Roll/pitch/yaw angular velocity 증분 각 축 |
|------------|---------------------|----------------------------------------|
| 0          | 기본 push 없음          | 없음                                     |
| 130,000    | $\pm 0.1$ m/s       | $\pm 0.2$ rad/s                        |
| 140,000    | $\pm 0.2$ m/s       | $\pm 0.4$ rad/s                        |
| 150,000    | $\pm 0.3$ m/s       | $\pm 0.6$ rad/s                        |
| 160,000    | $\pm 0.4$ m/s       | $\pm 0.8$ rad/s                        |
| 170,000    | $\pm 0.5$ m/s       | $\pm 1.0$ rad/s                        |

Deterministic play 및 현재 video transfer는 이런 events 없이 full gravity로 시작한다. 구현: envs/events.py, shared EventsCfg.



# 11. PPO 학습 하이퍼파라미터

새 PPO를 구현하지 않고 RSL-RL OnPolicyRunner /PPO를 사용한다. Floating config와 현재 10k run의 params/agent.yaml을 대조한 값이다.

| 항목                                     | 현재 floating 설정                                        |
|----------------------------------------|-------------------------------------------------------|
| Actor input / output                   | 67 / 12                                               |
| Critic input / output                  | 94 / scalar value                                     |
| Actor / critic hidden layers           | 각각 [1024,512,256,128]                                 |
| Activation                             | ELU                                                   |
| Actor mean 마지막 layer                   | zero initialization                                   |
| Observation normalization              | Actor/critic 각각 ON                                    |
| Action distribution                    | Gaussian, initial std 0.5, 저장 config의 std_type=scalar |
| Optimizer                              | Adam                                                  |
| Initial learning rate                  | $10^{-3}$                                             |
| LR schedule / desired KL               | adaptive / 0.01                                       |
| Rollout steps per env                  | 24                                                    |
| PPO epochs / mini-batches              | 5 / 4                                                 |
| Policy clip                            | 0.2                                                   |
| Value loss weight / value clipping     | 1.0 / ON                                              |
| Entropy coefficient                    | 0.002                                                 |
| Discount gamma / GAE lambda            | 0.998 / 0.95                                          |
| Max gradient norm                      | 1.0                                                   |
| Per-mini-batch advantage normalization | false (저장 run 설정)                                     |
| NaN check                              | true (저장 run 설정)                                      |
| Save interval                          | 500 updates                                           |
| Class max iterations                   | 20,000                                                |
| 현재 채택 run의 실제 요청 budget                | 10,000 updates, seed 42, 4096 env                     |
| Logger                                 | 해당 run은 TensorBoard                                   |

한 update의 batch size는  $N_{\text{envs}} \times 24$ 이다. 4096 env이면  $4096 \times 24 = 98,304$  transitions, mini-batch당  $98,304/4 = 24,576$  transitions이고 update당  $5 \times 4 = 20$  optimizer minibatch steps다. 10,000 updates의 설정상 총량은  $98,304 \times 10,000 = 983,040,000$  transitions다. 이는 10,000 episode나 시뮬레이션 frame이라는 뜻이 아니다.

학습은 rollout 수집 → critic/GAE return·advantage 계산 → clipped PPO actor/value loss update → checkpoint/log 저장 순서다. Gaussian sampling은 training용이고 deterministic evaluation은 frozen mean policy를 사용한다.

## Checkpoint 선택과 resume

기본 모델의 전체 상대 경로:

```
logs/rsl_rl/floating_revo2_tuna/  
2026-09-08_01-28-29_floating_stable_ground_10000/model_9999.pt
```

이름의 9999는 해당 완료 run의 저장 번호다. `scripts/_common.sh` 가 명시적으로 선택하며 가장 최근 폴더를 무조건 고르지 않는다. `--checkpoint` 또는 `REGRIND_FLOATING_CHECKPOINT` 로 override한다. Arm transfer/capture run은 floating 기본 checkpoint selection에 섞지 않는다.

저장된 `env.yaml` 의 reference 경로만 같다고 학습 당시와 현재 reference의 바이트까지 동일하다는 뜻은 아니다. 정확한 재현에는 당시 hash/보존된 입력과 대조해야 한다. 이 문서의 38-frame 수치는 현재 파일을 읽은 결과다.

현재 runner의 `--max_iterations N` 은 이번 호출에서 수행할 **update** 수다. Resume해도 절대 최종 iteration 번호가 아니다. 중단한 optimizer/normalizer를 복원하는 것과 PhysX contact·진행 중 episode·모든 RNG 상태까지 완전히 복원하는 것은 다르므로, split resume을 처음부터 연속 학습한 결과와 bitwise 동일하다고 보장하지 않는다.

기준: PPO config, train.py, ZeroInitMLPModel.

## 12. RB3에 정책을 연결하는 실제 실행

### 12.1 정책과 IK 사이의 frame

정책 wrist와 FK/IK endpoint는 모두 `right_hand_base_link`이다. RB3 link6, 일반 tcp, mount prim, semantic `kp_00`은 각각 다른 frame이다.

현재 모델의 fixed transform:

$$T_{\text{world,handbase}}(\mathbf{q}) = T_{\text{world,link6}}(\mathbf{q}) T_{\text{link6,handbase}}, \quad T_{\text{link6,handbase}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.141304972 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Translation 단위는 m이고 rotation은 identity다. Link6를 endpoint로 쓰는 외부 IK라면 target을 다음처럼 변환한다.

$$T_{\text{world,link6}}^{\text{target}} = T_{\text{world,handbase}}^{\text{target}} T_{\text{link6,handbase}}^{-1}$$

이 프로젝트의 FK/IK에는 위 fixed transform이 이미 포함되어 있다. 현재 solver에 handbase target을 넣으면서 mount를 또 빼면 이중 보정이다. 모델 설명상 link6에서 flange 끝까지 100 mm + adapter 41.304972 mm다. 과거 대화에서 추정한 34 cm를 현재 offset으로 쓰지 않는다.

책상 상대 world에서 RB3 설치 pose는  $(0, 0, -0.02)$  m, rotation identity다. Env origin translation을 제거한 target과 같은 base pose를 사용한다. USD root prim과 Revo2 branch 위치는 model JSON에 명시돼 있다.

### 12.2 현재 기본 video 경로

```
scripts/rl.sh play-arm
→ scripts/evaluate_mounted_interface.sh
→ tools/rb3_revo2_ik/evaluate_mounted_interface.py
→ configure_video_execution / select_video_arguments
→ FloatingObservationsCfg + FrozenPolicyAdapter
→ SimpleMountedWrist.process_actions
→ SimpleMountedWrist.apply_actions
→ named arm/hand joint position·velocity target
→ simulator write / physics step
→ runtime state → 다음 policy observation
```

Policy는 30 Hz에서 실제 관측으로 action을 계산한다. 손목 decoder target은 policy step 동안 유지하고, response shaping/IK/arm command는 120 Hz physics step에서 처리한다.

### 12.3 Wrist response shaping

현재 승인 config는 `response_tau=0.1 s`, `response_at_physics=True` 다.

$$\alpha = 1 - \exp\left(-\frac{\Delta t_{\text{physics}}}{\tau_{\text{response}}}\right) \approx 0.079956,$$

$$\mathbf{P}_{\text{shaped,next}} = \mathbf{P}_{\text{shaped}} + \alpha (\mathbf{P}_{\text{decoded}} - \mathbf{P}_{\text{shaped}}),$$

$$\Delta \theta = \text{rotvec}\left(Q_{\text{decoded}} \otimes Q_{\text{shaped}}^{-1}\right),$$

$$Q_{\text{shaped,next}} = \text{Quat}(\alpha \Delta \theta) \otimes Q_{\text{shaped}}.$$

회전은 shortest rotation 방향을 쓴다. 이는 **목표를 부드럽게 만드는 causal first-order response**다. 0.1 s 기다렸다가 한 번에 움직이는 고정 delay가 아니고, 측정된 손 상태나 별도 floating hand simulation을 대신하는 것도 아니다. 원래 decoder target에 대한 lag가 생길 수 있으므로 shaped target과 raw target 오차를 구분한다.

## 12.4 IK → 최종 actuator target

1. 기존 해를 warm start, 현재 실제 RB3 q를 neutral 후보로 전달한다.
2. WarmStartIK 가 같은 solver의 analytic-Jacobian local solve를 먼저 시도한다.
3. Strict pose/finite/limit 검증에 성공하고 이전 q 대비 각 joint 변화가 0.15 rad 이하이면 shortcut 결과를 사용한다. 아니면 기존 multiseed solver로 fallback한다.
4. 성공한 raw IK q를 goal로 두고, 실패하면 이전 accepted goal 유지 여부를 명시적으로 기록한다.
5. 실제 전달  $\mathbf{q}$ 는  $v_{\max} \Delta t_{\text{physics}}$ 의 raw coordinate step으로 제한하고 joint bounds도 적용한다. 모든 각도를 무조건  $\pm\pi$ 로 wrap하지 않는다.
6. 현재 video 경로는  $\mathbf{v}_{\text{target}}[k] = \frac{\mathbf{q}_{\text{cmd}}[k] - \mathbf{q}_{\text{cmd}}[k-1]}{\Delta t_{\text{physics}}}$ 를 계산해 speed limit으로 제한하여 같이 전달한다.
7. Revo2는 동일 policy timestamp의 6 leader target과 5 follower target을 사용한다. Recorded actual finger q를 새 target으로 바꾸지 않는다.

WarmStartIK의 0.15 rad는 최종 안전 step bound가 아니라 shortcut 채택 조건이다. 최종  $(10 \text{ rad/s}) \times (\frac{1}{120} \text{ s}) \approx 0.083333 \text{ rad}$  제한과 역할이 다르다. Raw IK jump가 남아도  $\mathbf{q}_{\text{cmd}}$ 는 다를 수 있다.

현재 default에는 velocity\_bounded\_ik=False, ik\_acceleration\_limit=0 이다. 뒤에 설명하는 acceleration-bounded 후보가 자동 적용됐다고 가정하지 않는다.

구현: SimpleMountedWrist, WarmStartIK.

## 12.5 초기화와 실제 상태 읽기

현재 기본 실행은 팔을 전부 편 상태에서 천천히 접근하는 homing 절차가 아니다. Reference/저장 bank의 arm·hand·object 상태와 phase를 복원하고 IK 초기화한다. Reference reset에서는 pose 선택과 실제 상태 write 후 IK warm start, target, response/velocity history를 동기화한다.

정책용 실제 wrist는 robot.data.body\_pos\_w/body\_quat\_w의 해당 handbase body이고, 실제 q/qdot는 runtime joint tensor다. Object는 runtime root state다. 모두 현재 timestep에 갱신된 PhysX-backed 경로를 사용하며 USD XformCache만으로 동적 pose를 읽었다고 간주하지 않는다.

오차는 다음을 분리한다.

```
A: 실제 IK 입력 target ↔ FK(q_ik)      # solver / pose approximation
B: FK(q_ik) ↔ FK(q_cmd)                # rate limit / 후처리 영향
```

```
C: FK(q_cmd) ↔ 실제 runtime handbase # 동적 추종  
별도: raw policy decoder target ↔ shaped target
```

목표가 30 Hz/120 Hz로 갱신된다는 것은 다음 갱신까지 반드시 도착한다는 뜻이 아니다. 또한 simulation time 1 s가 wall time 1 s와 같다는 보장도 없다. 느린 렌더링, CPU IK, GPU sync, recording 비용은 동작 시간계약과 구분해 측정한다.

## 13. 게인·접촉 설정과 floating/arm의 차이

### 13.1 현재 arm actuator 설정

ARM은 `ImplicitActuatorCfg`, force-type drive다. 아래는 코드/config에서 확인한 simulation 설정이며 제조사 사양이나 실물 튜닝 결과가 아니다.

| Joint    | Baseline Kp | Baseline Kd | 승인 video Kp | 승인 video Kd | Effort limit [N·m] | Velocity limit [rad/s] |
|----------|-------------|-------------|-------------|-------------|--------------------|------------------------|
| base     | 300         | 20          | 700         | 35          | 10                 | 10                     |
| shoulder | 500         | 20          | 20,000      | 260         | 100                | 10                     |
| elbow    | 500         | 20          | 12,000      | 140         | 100                | 10                     |
| wrist1   | 300         | 20          | 900         | 45          | 100                | 10                     |
| wrist2   | 200         | 20          | 2,400       | 70          | 100                | 10                     |
| wrist3   | 50          | 10          | 250         | 20          | 10                 | 10                     |

Kp 단위는 N·m/rad, Kd는 N·m·s/rad인 force joint drive 설정이다. Current video는 precision candidate `c3` 를 명시적으로 적용한다. 물리 effort/velocity limit을 늘려서 따라가게 한 설정은 아니다.

Revo2 leader/follower 모두 nominal Kp=3, Kd=0.1, effort limit=0.5 N·m, velocity limit=100 rad/s다. Floating training에 서만 앞의 mass/gain randomization 등이 적용된다. 손가락 자체를 arm precision benchmark와 동일하게 튜닝 완료했다고 주장하지 않는다.

현재 일반 실행에 검증된 state-dependent gravity torque feedforward를 추가해 사용하고 있지는 않다. `applied_torque` 라는 필드 이름만으로 implicit solver drive torque 측정값이라고 단정하면 안 된다. Drive-only provenance를 확보하지 못한 포화 판정은 **UNKNOWN**이다. 상세 근거는 actuator 진단을 따른다.

### 13.2 Tuna와 rubber 근사

| 대상                          | 설정                                                   |
|-----------------------------|------------------------------------------------------|
| Tuna                        | Dynamic rigid body, nominal mass 0.15 kg, gravity ON |
| Tuna visual / collision     | YCB textured mesh / bounds-derived cylinder collider |
| Tuna nominal friction       | static=0.8, dynamic=0.8, restitution=0               |
| Tuna contact/rest offset    | 0.002 m / 0                                          |
| 승인 mounted rubber 범위        | 5 distal link + fixed touch children                 |
| Rubber 근사 friction          | static=0.8, dynamic=0.8                              |
| Contact stiffness / damping | 10,000 N/m / 10 N·s/m                                |
| 실물 보정 여부                    | 미측정·미보정                                              |

revo2\_rubber\_contact.json은 **접촉 응답의 compliant 근사**다. 변형 가능한 고무 mesh/FEM 모델이 아니고 눌림 형상을 재현하지 않는다. 원본 USD geometry/mass를 덮어쓰는 대신 spawn 시 지정된 collision material을 적용한다. Friction을 과장해서 올린

실험도 아니다.

Floating 기본 train/play에는 이 contact override가 자동 적용되지 않는다. 따라서 현재 floating과 mounted를 비교할 때는 root control, gravity, arm·mount inertia, table geometry뿐 아니라 contact override 유무도 확인해야 한다.

### 13.3 Mounted workcell

rb3\_revo2\_table.json의 좌표는 tabletop-relative다.

| 항목                 | 값                              |
|--------------------|--------------------------------|
| Tabletop / floor Z | 0 / -0.72 m                    |
| 로봇 받침 크기 / center  | 0.5×0.5×0.7 m / (0,0,-0.37)    |
| 받침 상단·로봇 설치 Z      | -0.02 m                        |
| 책상 XY 크기 / center  | 0.8×1.6 m / (0.65,0)           |
| 책상 XY 범위           | X [0.25,1.05] , Y [-0.8,0.8] m |
| 상판 두께              | 0.04 m                         |

캔 origin 높이를 무조건 Z=0으로 놓으면 안 된다. Mesh 최저점이 Z=0이어야 한다. 현재 reference의 origin Z=0.012636 m는 이 구분 때문이다.

### 13.4 팔 실행을 개선해 온 과정

아래는 조건이 다른 과거 실험들의 요약이다. 행 사이를 동일 조건 A/B처럼 비교하지 않는다. 시간순 세부 근거와 재현 링크는 팔 제어 이력에 모아 두었다.

| 단계                          | 시도·관찰                                                                  | 배운 점 / 현재 의미                                              |
|-----------------------------|------------------------------------------------------------------------|-----------------------------------------------------------|
| 초기 combined RL              | Arm joint residual 6 + hand joint residual 6                           | 현재 Cartesian wrist residual 정책과 앞 6차원의 의미가 다름. Legacy로 구분 |
| Floating policy + online IK | 실제 mounted wrist/hand/object를 같은 의미의 관측으로 입력                           | 손목 target은 평형점이지 floating actual 궤적과 동일하지 않음              |
| Frame/mount·명령 진단           | FK/mount는 일치. 108개 고정 명령은 policy/IK 없이 실패를 재현                          | 프레임 오류와 동적 추종오차를 분리                                       |
| Can contact OFF             | C 최대 위치오차 약 51.234 mm가 남음                                              | 캔을 놓친 뒤 policy가 이상 행동한 것만으로 설명 불가                         |
| 속도 target만 0→차분             | OFF C 최대 51.234→29.839 mm. Live 20배치 성공은 19→7                          | 추종 개선과 가속·접촉 악화가 동시에 가능                                   |
| 정적/저속 gain benchmark        | c3 held-out 정지 최대 22.329→0.773 mm, 저속 P95 22.322→0.773 mm              | 고정된 부드러운 시험에서 정밀도 달성. 원속도 파지 보증 아님                        |
| 초기 simple interface·필터      | 과거 5k 정책 20배치: 기존 arm 19, simple 17, 30 Hz 필터만 추가 5                    | 인터페이스 단순화나 필터 하나가 자동 개선을 보장하지 않음                          |
| 결합 controller 검증            | c3 + 120 Hz response + bounded 최종 q 차분에서 old20/heldout20 lift proxy 통과 | 이후 승인 video 경로의 바탕. Raw policy target 오차와 가속 한계는 남음       |

| 단계              | 시도·관찰                                                  | 배운 점 / 현재 의미                                 |
|-----------------|--------------------------------------------------------|----------------------------------------------|
| 영상/최근 실행 parity | 영상과 최근 실행은 checkpoint뿐 아니라 controller/IK rate도 달랐음     | 저장 metadata로 실행 조건을 복원; 현재 shared config로 선택 |
| 고무·추가 학습        | Contact 근사 후 동일 controller fine-tuning, 기본 policy는 유지  | 다음 절의 별도 평가 결과로 판단                           |
| 특이점 대응          | 작은 wrist target에 raw q 급변. Branch-bounded IK·yaw 후보 비교 | Gain 문제와 별개이며 15절에서 strict/approx/충돌 조건 분리   |

회귀 테스트와 recorded replay가 중요한 이유는, 단일 “성공 영상”만 보고 어떤 변경이 효과가 있었는지 알 수 없기 때문이다. 현재 조합이 일부 배치에서 잘 잡아도 실물 안전 제어나 모든 배치의 성공을 의미하지 않는다.



# 14. Arm fine-tuning

이 경로는 기존 **floating** 정책 자체를 초기값으로 가져와 **mounted physics**에서 계속 **PPO** 업데이트한다. Frozen floating policy 뒤에 두 번째 residual policy를 붙이지 않는다.

| 항목                                | 현재 승인 video transfer                                                  |
|-----------------------------------|-----------------------------------------------------------------------|
| Observation/action/network/reward | Floating 67/94/12 계약 및 기존 PPO 그대로                                     |
| Arm controller                    | 평가와 같은 video response/IK/c3/material                                  |
| Env 수                             | <b>1만 지원</b>                                                          |
| Initial placement                 | 고정, 도달 가능한 배치                                                         |
| RSI / randomization / curriculum  | OFF / OFF / OFF                                                       |
| Gravity                           | full $-9.81\text{ m/s}^2$                                             |
| Fresh transfer                    | actor/critic/distribution/normalizer load, optimizer NEW, iteration 0 |
| Initial learning rate             | $10^{-4}$ , adaptive schedule 유지                                      |
| Save interval                     | 25 updates                                                            |
| Normalizer                        | Training에서는 갱신, evaluation에서는 frozen                                  |
| Resume                            | 해당 transfer optimizer·학습 counter·모델 복원, physical episode는 새 reset     |
| Log root                          | logs/rs1_rl/rb3_revo2_tuna_transfer_video/                            |

기존 baseline transfer는 병렬 smoke/RSI 경로를 보존하지만, 그것이 승인 video controller에서도 16/4096 env가 지원된다는 뜻은 아니다. 기본 video transfer에 RSI나 다중 env를 요청하면 명시적으로 거부한다.

arm\_transfer.py는 로드 직후 deterministic action parity, optimizer 초기화/복원, finite gradient/loss, weight 변화와 원본 hash를 확인한다. Controller/material/reference contract mismatch도 검사한다. `--training-seconds` 는 simulator startup을 제외하고 complete PPO update 이후 budget을 검사하므로 정확히 그 초에서 physics를 강제 중단하지 않는다.

## 기존 실행 결과의 의미

- 승인 video/rubber + 원본 10k 정책: 저장 old20 + heldout20에서 task/lift proxy 40/40을 기록했다.
- 같은 조건의 1시간 transfer: 2,679 updates, 64,296 transitions, 약 3,601 s. `model_2678.pt` 를 별도 저장했다.
- 해당 비교에서도 40/40을 유지했지만 평균 object error는 3.94→11.16 mm, finger error는 0.02235→0.02880 rad로 증가했다.
- 따라서 fine-tuned model을 기본으로 승격하지 않았다. 40개의 유한 배치 결과는 전체 workspace의 성공률 보장이 아니다.

상세 조건·로그·체크포인트는 RL\_TASK의 1시간 실험을 참조한다. 다른 baseline/material로 수행한 과거 100-update 실험과 숫자를 섞지 않는다.

# 15. 특이점·책상 작업영역 실험

## 15.1 문제 구간과 opt-in 수정

기존 기록 placement11에서는 손목 target이 2.317 mm / 0.002692 rad 움직일 때 raw IK의 wrist1/wrist3가 약 -0.798/+0.798 rad 급변했다. Wrist2≈0.9°로 두 wrist 축이 거의 일치했다. 관절 끝 limit이나 팔 완전 신전 문제라는 근거는 없었다.

별도 후보는 기존 least-squares/Jacobian을 유지하면서, **solver 내부 탐색 범위를** previous command 기준 position/velocity bounds와 교차시켰다. 이후 command acceleration bound 250 rad/s<sup>2</sup>도 시험했다. 이는 제조사 limit이 아닌 실험용 명령 제약이다.

| 항목                        | 후보의 의미                                                                   |
|---------------------------|--------------------------------------------------------------------------|
| Strict IK metric          | 여전히 0.1 mm / 0.001 rad로 별도 기록                                            |
| 초기 approximate budget     | 5 mm / 0.05 rad                                                          |
| Acceleration-bounded 동작   | 급정지 대신 수렴한 feasible progress를 사용하며 pose budget 초과는 별도 기록; 항상 예산 만족 보장 아님 |
| 기존 placement11 offline 결과 | max step 0.083333 rad, max pose error 0.425 mm / 0.01412 rad             |
| Historical live 40배치      | task/contact proxy 유지, sampled wrist max 약 6 mm 수준                       |
| 현재 기본값 여부                 | <b>아님</b> , 명시적 옵션                                                       |

자세 오차를 허용한 결과를 정확히 같은 SE(3)를 완벽히 추종한 것처럼 설명하지 않는다. Gain 튜닝은 힘/응답을 바꾸지만 특이점 자체를 제거하지 않는다. Static/slow precision benchmark의 1 mm 성능도 원속도 closed-loop grasp 전체에 대한 보증이 아니다.

설명·재현: 특이점 개선, 정밀추종 benchmark, 팔 제어 개발 이력.

## 15.2 작업영역 / branch / yaw 선택

최근 offline 도구는 책상 XY grid에서 전체 reference를 IK로 풀고, position·orientation tolerance, joint speed, 근사 workcell clearance를 검사한다. 초기 자세 후보, 앞/뒤 방향 branch 탐색, sequence graph 선택으로 큰 joint jump를 줄이는 실험도 있다.

원형 캔의 편한 접근 방향을 고르는 yaw 실험에서는 **캔을 중심으로 손·물체 reference 전체를 함께 회전**시킨다. 손목 orientation 하나만 돌리는 것이 아니다. 기존 orange 249곳에서 선택된 yaw로 strict IK/speed/근사 clearance 통과를 기록했지만 다음은 아직 별개다.

- 실제 mesh-level collision / self collision / 초기 접근 경로 검증
- 모든 위치의 acceleration·effort·closed-loop grasp 평가
- yaw를 반영한 실제 정책 observation/action의 canonicalization

따라서 이 결과를 “책상 249곳에서 실제 캔 파지 성공”이나 “현재 default가 자동 최적 yaw를 사용”한다고 표현하면 안 된다. 상세 결과와 한계: TABLETOP\_OPERATING\_REGION.md.

## 15.3 2026-09-11 고정 목표 궤적 비교

별도 실험은 예전 실패 episode 11의 **기록된 실제 IK 입력**을 같은 120 Hz 시간축에서 비교했다. 최신 10k 정책을 다시 실행한 결과가 아니다. 아래 오차는 IK 후보의 FK 대 목표이며 actuator 추종오차가 아니다.

| 방법                                                 | 최대 단일 관절 step<br>[rad] | 최대 명령 속도<br>[rad/s] | 의미 / 제한                                             |
|----------------------------------------------------|------------------------|---------------------|-----------------------------------------------------|
| 기존 strict warm-start                               | 0.798175               | 95.781              | 손목 축 정렬 근처에서 큰 관절 변화                                |
| 관절 끝 limit를 0.1 rad 축소                             | 0.798175               | 95.781              | 해당 문제는 줄이지 못함                                       |
| Workcell 검사 없는 초기 branch 변경                        | 0.018901               | —                   | 로봇 중심선이 구조물을 통과해 <b>채택 불가</b>                       |
| Workcell 검사 포함 branch 변경                           | 0.128090               | 15.371              | 개선되지만 10 rad/s 명령 속도 조건 초과                          |
| Velocity-bounded IK                                | 0.083333               | 10.000              | 최대 FK 오차 0.425 mm / 0.809°                          |
| 위 방법 + command acceleration 250 rad/s <sup>2</sup> | 0.083333               | 10.000              | 최대 FK 오차 1.077 mm / 2.150°; strict 아님               |
| 전체 task yaw +60° 및 초기 branch 선택                    | 0.010733               | 1.288               | strict FK 및 근사 clearance 통과; <b>목표 접근 방향 자체가 다름</b> |

마지막 후보는 raw step을 약 98.66% 줄였고, 고정된 +60°로 추가 episode 0/5/19의 offline 검사도 통과했다. 하지만 centerline/AABB 검사는 mesh·self collision 검사가 아니며, 실제 파지 성공을 검증하지 않았다. 이 결과 때문에 기본 실행을 바꾸지 않았다. 자세한 조건·입력·재현은 특이점 비교 보고서를 참고한다.

눈으로 비교할 때는 일반 구간 영상과 특이점 근처 영상을 볼 수 있다. 이는 과거 기록의 **actual joint/object state**를 재생한 0.25배 속 시각화이며 새 physics 평가가 아니다. Raw IK가 0.798 rad 변했다고 실제 팔도 같은 tick에 0.798 rad 움직인 것은 아니다.

## 16. 대표 실행 명령

모두 저장소 root에서 실행한다. 아래는 **사용 예시**이며 문서 작성 중 학습/GUI/데이터 재생성을 실행한 명령은 아니다. Output 디렉터리는 기존 결과와 겹치지 않게 새 이름을 사용한다.

### 16.1 전처리 → 리타게팅 → world → reference

```
./scripts/run_pipeline.sh --sequence 20200709_143747_left
```

첫 전처리는 여러 dataset을 열거하고 지정 sequence의 기본 앞 12 frame trim을 수행한다. Stable RL reference를 자동 재생성하지 않는다. 기존 결과를 강제로 갱신할지 결정하기 전에 data pipeline을 확인한다.

### 16.2 FK / 단일 frame retargeting

```
PYTHONPATH="$PWD/regrind/source/regrind" \  
/home/wanjunkim/IsaacLab/.venv/bin/python regrind/scripts/retarget_hand_object.py \  
--robot revo2 --object tuna_fish_can --demo-type dexycb \  
--demo outputs/preprocessed/dexycb/20200709_143747_left/dexycb_right_hand_preprocessed.npz \  
--input-quat-convention wxyz --solver clarabel --penetration-tolerance 0.002 \  
--single-frame 0 --no-visualize --out outputs/retargeted/single_frame_review.h5
```

Standalone FK API:

```
import numpy as np  
from tools.revo2_kinematics.revo2_kinematics import Revo2Kinematics  
  
fk = Revo2Kinematics()  
q = np.zeros(6)  
points_in_hand_base = fk.get_keypoints(q)  
lower, upper = fk.get_joint_limits()  
assert points_in_hand_base.shape == (21, 3)  
assert np.isfinite(points_in_hand_base).all()
```

### 16.3 Floating zero / play / training

```
# 선택한 stable reference + residual=0의 물리 동작  
./scripts/rl.sh zero --gui --real_time  
  
# 기본 10,000-update floating policy, GUI  
./scripts/rl.sh play --real_time  
  
# XY random placement에서 실제 floating policy 관측으로 평가  
./scripts/rl.sh play --random-placement --real_time
```

```
# 짧은 PPO smoke
./scripts/rl.sh train --num_envs 16 --max_iterations 2 --headless \
  --logger tensorboard --run_name workflow_smoke

# 새 10,000-update full training
./scripts/rl.sh train --full --num_envs 4096 --max_iterations 10000 --headless \
  --logger tensorboard --run_name floating_new_10000

# TensorBoard는 별도 터미널에서
tensorboard --logdir logs/rsl_rl/floating_revo2_tuna
```

W&B는 기존 `--logger wandb --log_project_name NAME` 경로를 사용한다. 실행 환경의 로그인/설정은 별도다. 새 capture experiment 이름에는 `agent.experiment_name=NAME` override를 사용한다. 현재 `--experiment_name` flag는 updater에서 적용하지 않는 알려진 별도 이슈가 있다.

## 16.4 Offline export와 online arm 평가

```
./scripts/rl.sh play --headless \
  --rollout-path outputs/floating/workflow_review/rollout.h5
./scripts/floating_to_rb3.sh \
  --rollout outputs/floating/workflow_review/rollout.h5 \
  --out outputs/floating/workflow_review/reference_12dof.h5

# 현재 승인 controller + original 10k policy, 저장된 배치 bank
./scripts/rl.sh play-arm --episodes 20

# 이전 strict-IK baseline을 명시적으로 재현
./scripts/rl.sh play-arm --arm-controller baseline --num_envs 1 --real_time
```

기본 video 평가의 20개 배치는 저장 bank에서 복원한 상태다. 매번 새 uniform XY를 추출했다는 뜻이 아니다. 일반 12-DoF replay의 입력/kinematic/object physics 선택은 ISAAC\_SIM\_REPLAY.md를 따른다. 대표 launcher는 `./scripts/run_isaac_replay.sh` 다.

## 16.5 같은 arm controller에서 fine-tuning / resume / 비교

```
FLOATING=logs/rsl_rl/floating_revo2_tuna/2026-09-08_01-28-29_floating_stable_ground_10000/model_9999.pt
STATES=outputs/diagnostics/arm_transfer_recovery/heldout_initial_states_v2.json

# 새 transfer: fresh optimizer, 현재 승인 video controller, 1 env
./scripts/rl.sh train-arm --transfer-init "$FLOATING" --num_envs 1 \
  --max_iterations 100 --headless --run_name workflow_video_transfer

# 생성된 실제 checkpoint 경로로 바꿔 사용
TRANSFER=logs/rsl_rl/rb3_revo2_tuna_transfer_video/YOUR_RUN/model_N.pt
./scripts/rl.sh train-arm --resume --checkpoint "$TRANSFER" --num_envs 1 \
  --max_iterations 1000 --headless --run_name workflow_video_resume

# 아래 BEFORE/AFTER는 같은 controller/reference/state bank로 새 경로에 저장
```

```
./scripts/evaluate_mounted_interface.sh --mode simple --arm-controller video \
  --transfer-evaluation --checkpoint "$FLOATING" --states "$STATES" \
  --episodes 20 --headless --output outputs/diagnostics/workflow_pair/before
./scripts/evaluate_mounted_interface.sh --mode simple --arm-controller video \
  --transfer-evaluation --checkpoint "$TRANSFER" --states "$STATES" \
  --episodes 20 --headless --output outputs/diagnostics/workflow_pair/after
./scripts/analyze_transfer_recovery.sh outputs/diagnostics/workflow_pair/comparison \
  --transfer-before outputs/diagnostics/workflow_pair/before \
  --transfer-after outputs/diagnostics/workflow_pair/after
```

Fresh transfer에서 `--resume` 를 사용하지 않는다. 평가 bank를 checkpoint 선택/튜닝에 사용했다면 held-out이라고 보고하지 말고 별도 bank를 확보한다. 같은 seed뿐 아니라 실제 초기 arm·hand·object state와 phase를 비교한다.

## 16.6 발표용 시각화

```
bash scripts/record_retargeting_presentation.sh \
  outputs/visualizations/presentation/workflow_retargeting \
  --sequence 20200709_143747_left --hide-model-keypoints
```

이 명령은 MANO/Revo2 skeleton과 실제 USD hand를 함께 보여주는 **kinematic render**다. RL/물리 파지 영상이 아니다. Floating/arm 비교와 parallel training capture는 presentation 명령 모음을 사용한다. 영상의 0.5× 표시·마지막 frame hold·출력 fps는 학습 physics dt나 policy Hz를 바꾸는 설정이 아니다.

# 17. 출력 파일·검증·남은 한계

## 17.1 주요 출력

```
outputs/preprocessed/dexycb/<sequence>/
  dexycb_right_hand_preprocessed.npz
outputs/retargeted/dexycb/<sequence>/
  revo2_retargeted.h5
outputs/isaac/dexycb/<sequence>/
  world_trajectory.h5
  rb3_revo2_reference.h5
  rb3_revo2_reference_stable.h5      # 별도 관리, 자동 갱신 아님
outputs/visualizations/dexycb/<sequence>/
  ... HTML, manifest
outputs/floating/<run>/
  ... rollout.h5, reference_12dof.h5
outputs/diagnostics/<run>/
  metadata.json, physics.jsonl, policy.json, plots/reports
logs/rsl_rl/floating_revo2_tuna/<run>/
  params/agent.yaml, params/env.yaml, model_*.pt, TensorBoard events
logs/rsl_rl/rb3_revo2_tuna_transfer_video/<run>/
  ... transfer checkpoints, transfer audit/summary
```

| 단계               | 중요한 배열                                                                                                               |
|------------------|----------------------------------------------------------------------------------------------------------------------|
| Retargeted       | robot_pos (T,3) , robot_quat (T,4) , robot_joints (T,6) , robot_keypoints (T,21,3) , object/MANO, solver diagnostics |
| World            | wrist_pos_world , wrist_quat_world , object_pos_world , object_quat_world , mano_joint_world , T_world_camera (4,4)  |
| Final reference  | rb3_joints (T,6) , revo2_joints (T,6) , reference_joints (T,12) , wrist/object pose, FK/IK error:success metadata    |
| Floating rollout | 실제 wrist/hand/object state, action/reference/phase 등; 실패 후 autoreset episode와 이어 붙이지 않음                              |

현재 RL reference loader는 rb3\_joints 와 revo2\_joints 를 모두 요구하고, reference\_joints 도 있으면 concatenation 일치를 확인한다. (T, 12) 하나만 있으면 모든 loader가 자동 분해해 준다고 가정하지 않는다. T>=2 , finite, quaternion, joint 이름/순서, 성공 metadata, control dt도 확인한다.

## 17.2 검증 순서

1. 파일 shape·단위·quaternion·hand convention·모델 source hash 확인.
2. Revo2 semantic FK/mimic, RB3 mount FK/IK 및 joint order 확인.
3. Actor 67 / critic 94 / action 12, decoder·normalizer·history/reset 계약 확인.
4. Physics에서는 실제 runtime state를 사용해 q target/actual과 wrist A/B/C 오차를 같은 timestep에 비교.

5. 동일 초기 상태·checkpoint·reference·controller/material로 before/after 평가.

6. Task success와 contact/lift/hold/drop를 구분하고 실패→성공, 성공→실패 배치를 함께 기록.

대표 회귀 검증:

```
./scripts/run_tests.sh
```

주요 보호 대상은 `test_revo2_kinematics`, `test_hand_mirroring`, `test_world_transform`, `test_rb3_kinematics`, `test_rb3_vertical_mount_usd`, `test_reference_trajectory`, `test_rb3_revo2_rl_reference`, `test_frozen_policy_adapter`, `test_mounted_response`, `test_arm_transfer`, `test_warm_start_ik`, `test_velocity_bounded_ik`, `test_checkpoint_launchers` 다. Script wrapper는 shell syntax와 root `tests/` 를 검사한다. `regrind/source/regrind/test/` 의 Drake/package tests는 별도다.

Pure-Python 테스트 성공만으로 Isaac contact, grasp 성공 또는 GPU runtime 정상이라고 보고하지 않는다. 학습 검증에는 실제 finite reward/loss, gradient/update, weight 변화, checkpoint save/reload가 필요하다.

## 17.3 현재 확실히 구분해야 할 한계

- Floating에서 잘 잡아도 mounted에서 같은 경로가 보장되지 않는다. Root 제어·gravity·관성·접촉·작업대·IK/rate limit이 다르며, 실제 관측이 달라져 다음 policy action도 달라진다.
- 30 Hz/120 Hz 명령 전달, numerical IK precision, 실제 actuator precision은 서로 다른 지표다.
- 캔 symmetry는 현재 RL reward에 반영하지 않았다. Offline yaw search가 live policy에 자동 통합된 것은 아니다.
- General pipeline과 현재 stable reference의 출처·frame 수가 다르다. 비교 영상에도 사용한 reference를 기록해야 한다.
- 1시간 transfer 결과를 성능 향상으로 단정하지 않았다. 기본 모델은 원본 floating 10k다.
- Material compliance·gain·velocity/effort 값은 simulation 근사다. 실물 calibration, 센서 지연, 토크 provenance, 접촉 보정과 실제 deployment는 남아 있다.
- 원본 LEAP/WUJI·legacy combined arm task는 보존돼 있으나 최근 Revo2 경로의 검증을 그 경로의 성능 검증으로 확장하지 않는다.
- Keypoint JSON 두 복사본, 로컬 절대경로 metadata, 과거 report의 `not_default` 표기 등 관리상 주의점이 있다. 실행 선택은 현재 launcher와 shared config가 우선이다.

이 문서의 숫자는 확인 시점의 snapshot이다. 앞으로는 **소스/config → run의 저장된 params → runtime metadata → 실제 결과**를 함께 확인해 값이 언제·어느 경로에서 적용됐는지 구분한다. 더 짧은 탐색 지도는 architecture, 상태 구분은 current-status, 세부 운영은 RL\_TASK, DATA\_PIPELINE, ISAAC\_SIM\_REPLAY를 참조한다.

## 17.4 자료의 검증 범위

이전 워크플로우 문서 작성에서는 standalone FK (21,3)·finite, strict reference loader의 38 frame / 30 Hz / phase metadata, CLI help를 확인했다. 그때의 158-test 기록은 과거 snapshot이며 최근 추가된 특이점 테스트가 포함된 현재 개수와 다르다.

이번 공부 자료 확장에서는 `./scripts/run_tests.sh` 를 다시 실행해 **166 tests passed**, root shell syntax 검사를 확인했다. 소스·config·학습 입력을 바꾸지 않고 원본 코드 대조, 수식 렌더링, 문서 링크·예제·읽기용 파일을 검사했다. 구체적인 문서 검증 결과는 읽기용 산출물 옆 `validation.json` 에 저장한다.



물리 파지·학습·실물 검증을 새로 실행한 작업이 아니다. 본문의 실험 숫자는 조건과 출처가 적힌 **기존 기록**이다. Markdown과 탐색 링크를 갱신하고 HTML 읽기용 사본을 생성했으며, 기존 사용자 변경·source·config·checkpoint·reference·USD는 보존했다.

## 18. 원본 논문·공개 코드와 무엇이 다른가

### 18.1 비교 기준과 출처

REGRIND의 핵심은 사람-물체의 기하학적 관계를 리타게팅으로 보존하고, 이를 **reference**로 삼아 물리 시뮬레이션에서 **residual policy**를 학습하는 것이다. 원 논문은 LEAP/WUJI와 가위·드라이버 예제를 다루며, floating 학습과 arm-mounted 실행을 구분한다. 본 프로젝트의 Revo2/RB3/DexYCB/tuna 조합 자체가 논문의 실험 조건은 아니다. 논문 v1, 공식 프로젝트.

공개 코드 비교는 2026-09-11에 확인한 upstream commit `38347a9e30184620df04e19c63c7c72378cae103` 를 기준으로 한다. 해당 커밋 기록 시각은 2026-07-14 UTC다. 현재 로컬 HEAD는 `e452939` 지만 작업 트리에 이후 변경도 존재하므로, 로컬 내용을 그 커밋 하나와 동일하다고 표시하지 않는다. 전체 upstream 이력을 복제하지 않고 관련 파일·함수만 비교했다.

| 자료                     | 답해 주는 질문                  | 답하지 못하는 질문                  |
|------------------------|---------------------------|-----------------------------|
| 논문                     | 왜 이 방법을 쓰는가, 어떤 실험을 보고했는가 | 현재 로컬 launcher가 어떤 값을 선택하는가 |
| 고정 upstream 코드         | 어떤 알고리즘·함수를 재사용했는가        | 현재 RB3 asset의 실제 drive 설정   |
| 로컬 source/config       | 어떤 경로와 값이 구현돼 있는가         | 물리 실행 없이도 실제 파지가 되는가        |
| Run params-runtime log | 해당 실행의 상태·값·결과            | 다른 모델·배치에도 동일한가             |
| 회귀 테스트                 | 수학·계약·경계 조건이 보존되는가        | 실물에서도 안전하고 안정적인가            |

논문에 보고된 실물 arm/controller 성능을 RB3 PhysX 설정에 그대로 적용하지 않는다. 원본 실물 제어기 내부 구현·정확한 gain·모든 timing을 공개 코드에서 확인했다고 가정하지 않는다.

### 18.2 변경 지도

앞 절의 설정표가 값의 기준이다. 여기서는 숫자를 반복하지 않고 **무엇을 유지했고 무엇을 대응시켰는지** 비교한다.

| 영역               | 원본에서 재사용한 것                                             | 이 프로젝트에서 추가·변경한 것                                                        | 위치 / 주의                                                     |
|------------------|---------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------|
| 데이터              | 사람 손·물체 pose를 reference로 쓰는 구조                          | DexYCB 두 번째 카메라, 유효 frame, tuna, 오른손 convention                          | <code>tools/dexycb_batch/</code> ; object mesh는 mirror하지 않음 |
| 손 모델             | Robot-specific constants                                | Revo2 6 leader, 11 moving joints, JSON 21 점·local offsets                | <code>retargeting/revo2_constants.py</code>                 |
| Interaction Mesh | Delaunay adjacency와 uniform Laplacian                   | Revo2 FK 점을 기존 objective에 연결                                             | <code>drake_utils.py</code> 는 비교 upstream과 텍스트 동일           |
| 최적화              | Drake 반복 선형화, quadratic 비용, joint/collision-temporal 제약 | Independent joint 변수화, mimic chain rule로 Jacobian 투영                     | <code>retargeter.py</code>                                  |
| Solver           | Mosek / Clarabel 경로                                     | Batch에서 Clarabel 명시, 입력·실패 진단 확장                                         | 새 REGRIND solver를 만든 것이 아님                                  |
| 실패 기록            | Frame별 retargeting                                      | <code>breakpoint()</code> 대신 예외, failure frame-objective-finite/limit 기록 | 실패를 이전 frame으로 숨기지 않음                                       |
| 좌표계              | Pose/point transform                                    | DexYCB camera→table world, quaternion metadata, 공통 placement             | <code>tools/dexycb_world_transform/</code>                  |

| 영역               | 원본에서 재사용한 것                                                        | 이 프로젝트에서 추가·변경한 것                                           | 위치 / 주의                                                                 |
|------------------|--------------------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------|
| FK/IK            | 손 keypoint의 기구학적 표현                                                | GUI 독립 Revo2 FK, RB3 model-mount-strict IK-warm-start       | <code>tools/revo2_kinematics/</code> , <code>tools/rb3_revo2_ik/</code> |
| 물체               | Object keypoint tracking                                           | Scissors joint 항목 비활성화, rigid tuna 50 local points          | <code>objects/tuna_can.py</code> , task config                          |
| Actor            | Object/hand state, history, reference/phase, previous action       | 67차원, mounted actual wrist, XY translation canonicalization | <code>mdp/observations.py</code>                                        |
| Critic           | 추가 velocity·fingertip 정보                                           | 94차원, 실제 5 touch body, articulated object joint 제외          | 21 semantic points 전체가 아님                                               |
| Action           | Wrist SE(3)+손 joint residual, reference 결합                         | Revo2 leader 6개 대응, 총 12차원, follower coupling               | RB3 joint residual 정책으로 바꾸지 않음                                          |
| Reward           | Object keypoint-velocity, wrist, action magnitude/rate exponential | Rigid-object config와 Revo2 이름·상태 대응                         | 주요 reward 함수 본문은 upstream 과 동일                                          |
| RSI              | Reference 임의 phase reset                                           | Combined loader, mimic-mounted IK-history·buffer reset 동기화  | <code>mdp/rb3_revo2_commands.py</code>                                  |
| Randomization    | Dynamics/observation lag, gravity/push curriculum                  | Embodiment별 범위, XY placement, 새 Isaac API                   | 모든 범위가 원본 손과 같지는 않음                                                     |
| PPO              | RSL-RL PPO, network 폭, 주요 하이퍼파라미터                                  | Task 등록, actor/critic config API 포팅, checkpoint 분리          | <code>config/revo2_floating/agents/</code>                              |
| Floating physics | Cartesian impedance PD 식                                           | Isaac 6 wrench API, low-inertia root에 대한 torque 분배          | 같은 총 torque여도 동역학 동등성은 별도                                               |
| Mounted control  | Policy residual이라는 상위 계약                                           | Actual 관측→동일 decoder→shaping→RB3 IK→actuator                | <code>mdp/simple_mounted_interface.py</code>                            |
| 제어 진단            | 논문의 새 알고리즘 아님                                                      | Frame 검증, A/B/C 분리, contact ON/OFF, gain-velocity 비교        | <code>tools/arm_diagnostics/</code>                                     |
| Fine-tuning      | 기존 PPO/정책을 계속 학습                                                   | 동일 arm config에서 floating checkpoint 초기화·전이 학습               | 별도 run; 현재 승인 경로 1 env                                                  |
| 특이점 대응           | 기존 FK/Jacobian 재사용                                                 | Warm solve, bounded IK, sequence branch, task yaw 탐색        | Offline 후보와 현재 기본값 구분                                                   |
| 운영/시각화           | 원본 source·LICENSE 보존                                               | Root launcher, HDF5/NPZ, HTML/영상, 경계 회귀 테스트                 | Physics와 actual-state 녹화 재생 구분                                          |

`retargeting/` 와 `mdp/` 는 각각 `regrind/source/regrind/regrind/retargeting/` 및 `regrind/source/regrind/regrind/tasks/manager_based/dexterous/mdp/` 의 약칭이다.

## 18.3 직접 확인한 재사용 증거

- Upstream drake\_utils와 로컬 파일은 텍스트가 같았다. Mesh·adjacency·Laplacian을 새로 발명한 것으로 설명하지 않는다.
- Upstream retargeter의 iteration 구조는 유지됐다. Single iteration의 주요 차이는 mimic 적용·Jacobian 투영·진단/예외 처리다. 첫 pose의 cone 해제와 비활성 backtracking도 원본에서 온 동작이다.

- Upstream reward의 keypoint, object velocity, wrist pose, action magnitude/rate/bounds 함수 본문은 같았다. Config와 asset까지 같다는 뜻은 아니다.
- Upstream PPO config의 network 크기·ELU·초기 std·zero mean 초기화·주요 PPO 값을 현재 floating에서도 유지한다. `policy=...` 가 `actor/critic=...` 로 나뉜 API 변경은 새 network 제안이 아니다.
- Upstream action의 `SE3ImpedanceActionTerm.process_actions` 본문도 동일했다. Quaternion helper/API 경계·wrench 전달은 포팅됐으므로 파일 전체가 같지는 않다.
- Upstream events의 observation-delay 초기화·lag 선택·random-push curriculum은 동일했고, gravity event는 manager API에 맞춰 stateful 형태로 바뀌었다.

논문·공개 구현·이 fork의 변경을 구분해야 한다. 예를 들어 논문 본문 mesh cost 표기는 L2 norm 합으로 설명되지만 공개 solver는 quadratic error cost를 사용한다. **Quadratic 구현을 이 프로젝트가 새로 바꾼 알고리즘으로 분류하면 틀린다.** 뒤의 유도는 로컬 실행 코드 기준이다. 논문 `3.1 및 부록.

## 18.4 버전 포팅은 단순한 import 변경이 아니다

원본 안내는 Python 3.11 / Isaac Sim 5.1 / Isaac Lab 2.3 계열이고 현재 설치는 2절의 값이다.

| Boundary      | 포팅 내용                                                      | 해석할 때 주의                                 |
|---------------|------------------------------------------------------------|------------------------------------------|
| Quaternion    | Legacy 저장 WXYZ ↔ 현재 runtime XYZW 변환                        | 모든 파일 열 순서를 한꺼번에 바꾼 것이 아님                |
| 상태 접근         | Warp-backed <code>.torch</code> view, pose/velocity API 분리 | Stale USD pose를 runtime actual이라고 읽지 않음  |
| 명령 전달         | 설치 버전의 indexed position/velocity target API                | 배열 index를 이름으로 대응                        |
| Root wrench   | Force는 root에, torque는 body mass 비율로 분배                     | 물리 응답이 동일한지는 별도 검증                       |
| RL model      | Actor/critic 의미를 RSL-RL 5 config로 표현                       | Checkpoint·normalizer·distribution 로딩 검증 |
| Gravity event | Manager term 초기화 방식 대응                                     | Counter·reset 시점·학습 budget 보존            |

상세 이력: Isaac Lab migration. 구현의 존재와 모든 원본 task의 최신 simulator 검증은 같은 말이 아니다.

## 19. 핵심 수식을 처음부터 연결하기

이 절은 구현을 이해하기 위한 유도다. 실제 배열 순서·단위·설정은 앞 절의 계약을 따른다.

### 19.1 Pose는 무엇이고 왜 곱하는 순서가 중요한가

$T_B^A$ 는 B 좌표의 점을 A 좌표로 옮기는 변환이다. 열벡터 convention으로 쓴다.

$$T_B^A = \begin{bmatrix} R_B^A & \mathbf{t}_B^A \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad \begin{bmatrix} \mathbf{p}^A \\ 1 \end{bmatrix} = T_B^A \begin{bmatrix} \mathbf{p}^B \\ 1 \end{bmatrix}, \quad T_C^A = T_B^A T_C^B.$$

$R$ 은 회전행렬,  $\mathbf{t}$ 는 m 단위 translation이다. 오른쪽부터 적용한다.  $R^T R = I$ ,  $\det R = +1$ 인 회전과 translation의 집합이 SE(3)다.

$$(T_B^A)^{-1} = \begin{bmatrix} (R_B^A)^T & -(R_B^A)^T \mathbf{t}_B^A \\ \mathbf{0}^T & 1 \end{bmatrix}.$$

Local 점이 (0.01, 0, 0) m이고 parent가 world (0.4, 0, 0.1) m에서 Z축 +90° 회전했다면 world 점은 (0.4, 0.01, 0.1) m다. Local xyz를 world translation에 그냥 더하면 회전이 빠진다.

원손 변환  $S = \text{diag}(-1, 1, 1)$ 은  $\det S = -1$ 이다. **Reflection은 rotation이 아니므로 quaternion 하나로 표현할 수 없다.** 현재 hand points만 object-local reflection하고, object mesh는 바꾸지 않는다. 이후 proper camera→world rotation과 구분한다.

Orientation은 quaternion 원소 차이가 아니라 회전각으로 비교한다. 단위 quaternion  $Q$ 와  $-Q$ 는 같은 회전이다.

$$\theta = 2 \arccos(\text{clip}(|\langle Q_1, Q_2 \rangle|, 0, 1)) = \|\text{Log}(R_1^T R_2)^\vee\|_2.$$

$\theta$ 는 rad, degree는  $180\theta/\pi$ 다. Near-zero 수치 처리는 기존 helper를 재사용한다.

Actor의 rotation 6D는 회전행렬 앞 두 column의 기존 row-major flatten이다.

$$\mathbf{r}_6(R) = [R_{00}, R_{01}, R_{10}, R_{11}, R_{20}, R_{21}]^T.$$

흔히 쓰는 “첫 column 3개 다음 두 번째 column 3개”와 순서가 다르다. 수치가 같아 보여도 순서를 바꾸면 기존 checkpoint에 다른 observation을 넣는다.

### 19.2 FK와 mimic의 미분

Leader  $\mathbf{u} \in \mathbb{R}^6$ 와 전체 moving joints  $\mathbf{q}_m \in \mathbb{R}^{11}$ 를 구분한다. Mimic은 affine 관계다.

$$\mathbf{q}_m = A\mathbf{u} + \mathbf{b}, \quad \begin{bmatrix} \mathbf{p}_i^W \\ 1 \end{bmatrix} = T_H^W T_{\text{parent}(i)}^H (A\mathbf{u} + \mathbf{b}) \begin{bmatrix} \mathbf{p}_{i,\text{local}} \\ 1 \end{bmatrix}.$$

$H$ 는 Revo2 base다. Link tree를 따라 fixed transform과 axis rotation을 곱하므로 leader 하나에 follower와 여러 keypoint가 함께 움직인다.

Jacobian은 “관절을 조금 움직였을 때 점이 얼마나 움직이는가”다. Chain rule에 의해:

$$\frac{\partial \mathbf{p}_i}{\partial \mathbf{u}} = \frac{\partial \mathbf{p}_i}{\partial \mathbf{q}_m} A, \quad J_{\text{leader},j}^{\text{effective}} = J_{\text{leader},j} + \sum_{\ell: \text{leader}(\ell)=j} m_\ell J_{\text{follower},\ell}.$$

FK에서는 follower를 움직이고 Jacobian에서는 빼면 solver가 잘못된 미분을 따른다. `_project_jacobian_to_independent` 가 이 문제를 막으며 signed-distance 미분에도 적용된다. 21점을 독립 변수로 만드는 방식이 아니다.

Affine mimic은 **기구학/target 관계**다. 유한 gain의 actual follower가 ideal mimic과 완전히 같다고 보장하지 않는다. 녹화 actual motion은 측정 follower와 ideal FK를 구분해서 검사한다.

### 19.3 Laplacian에서 실제 solver까지

사람 hand 21 + object 50점을  $V_h \in \mathbb{R}^{71 \times 3}$ 로 쌓는다. Delaunay tetrahedra로 이웃 집합  $\mathcal{N}(i)$ 를 얻는다. 현재 uniform Laplacian:

$$\delta_i = \mathbf{v}_i - \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \mathbf{v}_j, \quad L_{ij} = \begin{cases} 1 & i = j, \\ -1/|\mathcal{N}(i)| & j \in \mathcal{N}(i), \\ 0 & \text{otherwise.} \end{cases}$$

이웃이 없는 점은 구현상 row를 0으로 처리한다.  $LV_h$ 는 그 점이 주변 구조에서 차지하는 상대 위치다. 연결은 사람 mesh에서 정해 해당 최적화에 재사용한다.

로봇 vertices는  $V_r(\mathbf{x})$ , 목표는  $\delta_h = LV_h$ 이며 같은 object frame이다.  $\mathbf{x}$ 는 wrist quaternion 4 + position 3 + leader 6의 **13 scalar**다. 물리 자유도는 12지만 quaternion을 4성분으로 저장한다. Solver에는 아래 보조변수도 있다.

$\text{vec}_p$ 가 point별 xyz를 이어 붙이는 순서라 할 때:

$$\begin{aligned} \ell_n &= \text{vec}_p(LV_r(\mathbf{x}_n)) \in \mathbb{R}^{213}, \\ J_\ell &= (L \otimes I_3) J_V \in \mathbb{R}^{213 \times 13}, \\ \text{vec}_p(LV_r(\mathbf{x}_n + \Delta \mathbf{x})) &\approx \ell_n + J_\ell \Delta \mathbf{x}. \end{aligned}$$

Object rows의  $J_V$ 는 0이다. Hand rows만 FK+mimic chain rule로 움직인다. Auxiliary variable  $\mathbf{y}$ 는 선형화한 robot Laplacian이며 반복 subproblem은 다음 형태다.

$$\begin{aligned} \min_{\Delta \mathbf{x}, \mathbf{y}} \quad & \|\mathbf{y} - \text{vec}_p(\delta_h)\|_W^2 + \lambda_s \|\Delta \mathbf{x} - (\mathbf{x}_{\text{prev}} - \mathbf{x}_n)\|_2^2 + C_{\text{nominal}} \\ \text{subject to} \quad & \mathbf{y} = \ell_n + J_\ell \Delta \mathbf{x}, \\ & \mathbf{x}_{\min} - \mathbf{x}_n \leq \Delta \mathbf{x} \leq \mathbf{x}_{\max} - \mathbf{x}_n, \\ & d_c(\mathbf{x}_n) + J_{d_c} \Delta \mathbf{x} \geq -\varepsilon_c, \\ & \|\Delta \mathbf{x}\|_2 \leq s_{\text{step}}. \end{aligned}$$

$\|\mathbf{e}\|_W^2 = \mathbf{e}^T W \mathbf{e}$ 이고,  $d_c$ 는 signed separation,  $\varepsilon_c$ 는 penetration tolerance다. 값은 5.3절에 있다. 마지막은 second-order cone 제약이다. 첫 pose에서는 이 제약을 제거하는 예외가 있다.

해를 더한 뒤 quaternion 정규화와 mimic 적용을 하고 반복한다. 다음 frame을 warm-start하지만 전체 sequence의 global optimum은 보장하지 않는다. 혼합된 quaternion·m·rad의 trust bound와 temporal quadratic 비용은 물리 속도/가속도 제한이 아니다.

생각해 볼 예: 손·물체를 동일 translation으로 옮기면 object-frame 관계와 objective는 그대로다. 그러나 팔 base가 고정돼 있어 **RB3 IK 가능성은 바뀐다**. 리타게팅 가능 영역과 mounted 작업영역이 다른 이유다.

## 19.4 PPO는 무엇을 학습하는가

Policy는 residual  $\mathbf{a}$ 를 내고 actuator는 decoder·IK 이후 target을 받는다.

$$\begin{aligned}\hat{\mathbf{o}} &= \text{Normalize}(\mathbf{o}; \mu_o, \sigma_o), \\ \mathbf{a} &\sim \mathcal{N}(\mu_\theta(\hat{\mathbf{o}}), \text{diag}(\sigma_\theta^2)), \\ \mathbf{a}_{\text{eval}} &= \mu_\theta(\hat{\mathbf{o}}), \\ \text{target} &= \text{Decode}(\text{reference}[\phi], \text{clip}(\mathbf{a}, -1, 1)).\end{aligned}$$

Normalizer는 통계로 scale을 맞추는 것이지 world→object 좌표변환이 아니다. 실제 epsilon/clamp를 새로 만들지 말고 checkpoint의 정규화를 그대로 쓴다. Actor와 critic 통계도 별개다.

Actor는 이후 보상을 높이는 residual을, critic은 discounted return을 근사한다. Critic의 privileged 정보가 평가 actor의 추가 입력으로 들어가는 것은 아니다. Storage의 done indicator를  $d_t$ 라 할 때 GAE는:

$$\begin{aligned}\delta_t &= r_t + \gamma(1 - d_t)V_\psi(s_{t+1}) - V_\psi(s_t), \\ A_t &= \delta_t + \gamma\lambda(1 - d_t)A_{t+1}, \\ \hat{R}_t &= A_t + V_\psi(s_t).\end{aligned}$$

Timeout bootstrapping/autoreset은 기존 runner 계약을 따른다. 다음 episode 첫 관측을 이전 episode terminal 상태로 해석하지 않는다. 현재 per-mini-batch 정규화를 끄면 advantage를 **전체 rollout 기준으로 정규화**한다. 그 옵션이 false라고 정규화 자체가 없는 것이 아니다.

PPO ratio는 새 policy와 수집 당시 policy가 수집한 **action에 부여하는 확률 비율**이다.

$$\begin{aligned}\rho_t(\theta) &= \frac{\pi_\theta(\mathbf{a}_t | \mathbf{o}_t)}{\pi_{\text{old}}(\mathbf{a}_t | \mathbf{o}_t)}, \\ L_{\text{clip}} &= \mathbb{E}_t[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)], \\ V_{\text{clip}} &= V_{\text{old}} + \text{clip}(V_\psi - V_{\text{old}}, -\epsilon, +\epsilon), \\ L_V &= \mathbb{E}_t\left[\max\left((V_\psi - \hat{R}_t)^2, (V_{\text{clip}} - \hat{R}_t)^2\right)\right], \\ L_{\text{minimize}} &= -L_{\text{clip}} + c_V L_V - c_H \mathbb{E}_t[H(\pi_\theta)].\end{aligned}$$

현재  $\epsilon = 0.2$ 이며 나머지 값은 11절에 있다. Entropy는 탐색을 유지하는 항이다. **PPO ratio clipping, action  $\pm 1$  clipping, joint limit clipping**은 다른 단계다. 확률 모델을 “이미 잘린 Gaussian”으로 새로 바꾸지 않는다.

설치 RSL-RL의 `algorithms/ppo.py` 에서 위 GAE·global advantage normalization·clipped surrogate/value loss를 확인했다. 이 프로젝트는 해당 구현을 호출하며 별도 PPO를 작성하지 않는다. 배경은 PPO 원 논문을 참고한다.

RSI는 후반 frame의 좋은 상태도 바로 경험하게 하지만 처음부터 끝까지 자기 힘으로 성공시킨 평가와 다르다. 그래서 평가는 RSI OFF로 첫 frame부터 시작한다.

## 19.5 IK와 actuator는 서로 다른 문제다

IK는 어디에 있어야 하는가, actuator는 유한한 힘으로 어떻게 가는가를 푼다. 현재 RB3 pose residual은:

$$\mathbf{e}_{\text{IK}}(\mathbf{q}) = \begin{bmatrix} 10(\mathbf{p}_{\text{FK}}(\mathbf{q}) - \mathbf{p}^*) \\ \text{Log}((R^*)^T R_{\text{FK}}(\mathbf{q}))^\vee \end{bmatrix}, \quad \min_{\mathbf{q}_{\min} \leq \mathbf{q} \leq \mathbf{q}_{\max}} \|\mathbf{e}_{\text{IK}}(\mathbf{q})\|_2^2.$$

Position·rotation 단위가 달라 가중치를 준 것이다. 보고하는 실제 position error는 m이며 10으로 나눠 숨기지 않는다. Accepted 후보 중 이전 해에 가까운 쪽을 우선하지만 다중 해를 전부 발견한다는 보장은 없다.

Force-PD actuator를 이해하기 위한 연속시간 모델:

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}_{\text{drive}} + \boldsymbol{\tau}_{\text{contact}}, \quad \boldsymbol{\tau}_{\text{PD}} = K_p(\mathbf{q}_{\text{cmd}} - \mathbf{q}) + K_d(\dot{\mathbf{q}}_{\text{target}} - \dot{\mathbf{q}}) + \boldsymbol{\tau}_{\text{ff}}.$$

$M$ 은 질량행렬,  $C\dot{\mathbf{q}}$ 는 관성 관련 항,  $\mathbf{g}$ 는 중력항이다. 이 단순 모델과 실제 PhysX implicit 이산시간 solver를 동일시하지 않는다. 식으로 계산한 PD를 검증된 solver drive torque라고 기록할 수 있다는 뜻도 아니다.

Position target이 움직이는데 velocity target이 0이면 D항은 실제 이동에 반대한다.  $v_{\text{path}}$ 를 주면 지연을 줄일 수 있지만, 목표가 거칠면 acceleration/contact가 악화될 수 있다. 과거 20배치 단일 변경은 추종 개선과 파지 악화를 동시에 보였다. “속도 feedforward는 언제나 좋다/나쁘다”로 일반화할 수 없다.

아주 느린 평형에서는  $K_p \mathbf{e} \approx \mathbf{g}$ 라는 직관을 얻지만 coupling·contact·limits·IK target 오차가 섞인다. “오차는 무조건 gain”, “큰 오차는 torque saturation”으로 결론 낼 수 없다. Drive-only effort가 검증되지 않으면 saturation은 UNKNOWN이다.

## 19.6 Mount와 오차 분리

$B$ =RB3 base,  $E$ =link6,  $H$ =Revo2 base,  $W$ =world라고 하자.

$$T_H^W(\mathbf{q}) = T_B^W T_E^B(\mathbf{q}) T_H^E.$$

Link6 endpoint solver라면 다음 목표가 필요하다.

$$(T_E^B)^* = (T_B^W)^{-1} (T_H^W)^* (T_H^E)^{-1}.$$

그러나 현재 RB3 FK/IK는 이미 mount  $T_H^E$ 를 포함하고 endpoint가 Revo2 base다. 현재 함수에 mount inverse를 또 적용하면 frame mismatch가 된다. Semantic `kp_00` 역시 base origin과 다른 local point다.

| Pose          | 의미                             | 오차 비교에서의 역할    |
|---------------|--------------------------------|----------------|
| Policy target | Reference+residual, shaping 전  | 원래 정책 목표       |
| IK input      | Shaping 후 solver에 실제 전달한 목표    | A의 왼쪽          |
| FK(q_ik)      | Solver raw output의 FK          | A의 오른쪽 / B의 왼쪽 |
| FK(q_cmd)     | Mapping/limit 후 실제 command의 FK | B의 오른쪽 / C의 왼쪽 |
| Actual base   | 같은 physics clock의 runtime pose | C의 오른쪽         |



A는 IK 잔차, B는 후처리가 만든 차이, C는 command 대비 실제 추종 차이다. **A/B/C scalar norm을 더해도 일반적으로 total error와 같지 않다.** 방향과 회전 합성이 다르기 때문이다. Shaping 전→후 차이도 별도로 보고한다. C만 작아도 policy target을 크게 바꿨다면 원래 목표를 정확히 따른 것이 아니다.

### 19.7 작은 손목 움직임이 큰 관절 움직임이 되는 이유

국소적으로 task 변화와 joint 변화는 Jacobian으로 연결된다.

$$\Delta \mathbf{x} \approx J(\mathbf{q})\Delta \mathbf{q}, \quad J = U\Sigma V^T, \quad \Delta \mathbf{q} \approx J^\dagger \Delta \mathbf{x}.$$

Singular value  $\sigma_i$ 가 작으면 해당 방향 역산에  $1/\sigma_i$ 가 곱해진다. 손목 축이 거의 평행하면 회전을 상쇄하는 큰 joint 변화가 필요한 방향이 생긴다. Pose Jacobian에는 m/rad가 섞이므로 같은 가중치에서 singular value를 비교해야 한다.

| 문제                | 대표 증거                              | 현재 문제와의 관계                |
|-------------------|------------------------------------|---------------------------|
| Workspace 밖       | Position-only IK도 실패               | 모든 급변이 이것 때문은 아님          |
| Joint limit       | 허용 범위 경계·작은 margin                 | Episode 11 주요 원인이라는 증거 없음 |
| Wrist singularity | 축 정렬·작은 singular value·wrist1/3 상쇄 | 해당 기록에서 확인됨               |
| Branch 선택         | 다른 초기 해에서 완만한 posture              | 충돌/속도 검사 없는 후보는 채택 불가     |

감쇠된 역행렬은 개념적으로 큰 역수를 억제한다.

$$J_\lambda^\dagger = J^T(JJ^T + \lambda^2 I)^{-1}.$$

이는 배경 수식이며 **현재 기본 실행을 differential IK로 교체했다는 뜻이 아니다.** Damping은 pose error와 trade-off가 있고 충돌/접근 방향을 자동 해결하지 않는다. 실제 구현한 후보와 측정치는 15절에 있다.

원형 캔의 편한 접근 yaw는 중심 **c** 주변에서 **손·물체 reference 전체**를 바꾼다.

$$\mathbf{p}' = \mathbf{c} + R_z(\psi)(\mathbf{p} - \mathbf{c}), \quad R' = R_z(\psi)R.$$

캔 위치가 같아도 손의 초기 pose와 arm branch가 달라진다. 기하학적으로 편한 방향과 기존 policy의 회전 일반화는 별개다. Observation/action/history/reference의 일관된 yaw 처리와 contact 평가 전에는 offline 결과를 grasp 성공으로 확장하지 않는다.

## 20. 실제 구현 순서와 코드 읽기

다음은 모든 Git 시행착오의 연대기가 아니라 **현재 기능의 의존성 순서**다. 팔 제어 실험 이력은 별도 문서에 있다.

### 20.1 Dataset에서 reference까지

| 순서 | 실제 함수·파일                                                                                                                        | 입력 → 출력                         | 먼저 확인할 제약                          |
|----|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------|------------------------------------|
| 1  | <code>tools/dexycb_batch/preprocess_dataset.py::preprocess_sequence</code>                                                      | Label/meta → hand/object demo   | Camera, m, finite, object identity |
| 2  | 같은 파일 <code>_mirror_in_object_local_x</code>                                                                                    | Left points → right convention  | Object/mesh는 mirror하지 않음           |
| 3  | <code>tools/revo2_kinematics/revo2_kinematics.py::Revo2Kinematics</code>                                                        | Leader 6 → local 21 points      | Link/local xyz/mimic/index         |
| 4  | <code>regrind/scripts/retarget_hand_object.py</code>                                                                            | Demo+constants → solver 입력      | Initial wrist, joint order         |
| 5  | <code>HandInteractionMeshOneStageRetargeter.retarget_motion</code> → <code>iterate</code> → <code>solve_single_iteration</code> | Frame mesh+FK → wrist/hand      | 13 scalar, warm-start, 실패 기록       |
| 6  | <code>tools/dexycb_world_transform/transform_trajectory.py</code>                                                               | Camera → common world           | Pose·point 동일 변환, 캔 바 닥            |
| 7  | <code>RB3730Kinematics.inverse</code> / <code>forward</code>                                                                    | World wrist → arm 6 joints / FK | Mounted endpoint, XYZW             |
| 8  | <code>tools/rb3_revo2_ik/build_reference_trajectory.py</code>                                                                   | Arm+hand+object → HDF5/NPZ      | RB3→Revo2, phase/dt, failure mask  |

Standalone NumPy/Torch Revo2 FK와 Drake retargeting FK는 역할이 다르다. 전자는 GUI 없이 검사·시각화하기 쉽고, 후자는 optimizer의 geometry/Jacobian 경로다. 양쪽 계산과 USD runtime의 일치를 검증해야 한다.

아래는 실행 API가 아닌 읽기용 의사코드다.

```
load demo, keypoint metadata, robot model
validate indices / links / units / quaternion convention
last_success = initial wrist + six leader coordinates
for each frame:
    build human + object interaction mesh
    start from last_success
    repeat linearize / solve / normalize quaternion / apply mimic
    if valid:
        save wrist + six leaders + FK keypoints
        last_success = solution
    else:
        save explicit failure; do not fabricate a successful frame
apply one world transform to object, wrist and MANO
```

```
solve arm IK sequentially and verify same-frame FK
save arrays + timestamps + metadata
```

`run_pipeline.sh` 는 단순 검사 명령이 아니라 생성물을 작성한다. 기존 결과와 stable reference의 출처를 먼저 구분한다.

## 20.2 Floating 학습의 한 step

```
scripts/rl.sh train
→ regrind/scripts/rsl_rl/train.py
→ floating task / FloatingRevo2TunaPPORunnerCfg
→ ManagerBasedRLEnv + RSL-RL OnPolicyRunner
→ RB3Revo2ReferenceCommand / observations / actions / rewards
```

Command 이름에 `RB3` 가 있어도 floating은 손 reference-wrist body를 지정해 재사용한다. 이름만 보고 actor가 arm joints를 관측한다고 판단하면 안 된다.

| 역할                    | 확인할 구현                                                                               |
|-----------------------|--------------------------------------------------------------------------------------|
| Reference/phase/reset | rb3_revo2_commands.py: <code>_resample_command</code> , <code>_update_command</code> |
| Actual 상태             | 같은 command의 <code>current_*</code> properties                                        |
| 67/94차원 조립            | FloatingObservationsCfg, observations.py                                             |
| Wrist decode / wrench | actions.py: <code>process_actions</code> / <code>apply_actions</code>                |
| Leader/follower       | rb3_revo2_actions.py                                                                 |
| Reward/done           | 공통 config, mdp/rewards.py                                                            |
| PPO/model             | <code>train.py</code> → 설치 RSL-RL, actor_critic.py                                   |

```
observation(t): actual state + history + reference phase
policy: one action at 30 Hz
process_actions: clip → scale → reference combination, once
  repeat 4 physics steps:
    apply_actions: target → actuator/wrench
    write targets → PhysX advances 1/120 s → runtime state updated
reward / termination / transition storage at manager-defined timing
reset done environments; command / observation for next step
24 transitions per env collected → PPO update
```

엄밀한 reward/reset/command/observation 호출 순서는 설치된 `ManagerBasedRLEnv.step` 이 소유한다. 다음 관측의 frame 번호로 직전 physics command를 잘못 라벨링하면 한 frame 차이를 controller 지연으로 오해한다.

현재 phase는 기본적으로 다음 식이다. Trim과 원래 phase 관계는 metadata/override를 따른다.

$$\phi = \text{clip} \left( \frac{f_{\text{local}} + f_{\text{offset}}}{\max(T_{\text{phase}} - 1, 1)}, 0, 1 \right).$$

따라서 배열 길이만 바꾸거나 앞부분을 자르면 학습한 phase 의미가 달라질 수 있다. Reference trim·학습 continuation·영상 배속은 서로 다른 조작이다.

## 20.3 Mounted 실행에서 바뀌는 부분

Decoder 전까지의 의미는 floating과 같다. Actual state의 embodiment와 wrist target 실행 방식이 달라진다.

```
scripts/rl.sh play-arm --arm-controller video
→ shared arm_execution_config.py
→ mounted task + 동일 FloatingObservationsCfg
→ actual Revo2 base / leader joints / dynamic object
→ 기존 policy & normalizer
→ SimpleMountedWrist.process_actions (기존 decoder 1회)
→ 매 physics tick:
    response_step (tau=0.1 s, causal target shaping)
    solve (검증된 RB3 FK/IK, mount 이미 포함)
    position/speed target 제한
    arm + leader/mimic actuator command
    physics → actual state → 다음 30 Hz observation
```

Config 선택은 arm\_execution\_config.py, 실행은 SimpleMountedWrist에 있다.

- `process_actions` : inherited decoder로 목표를 한 번 만든다. Actual observation에 desired를 넣지 않는다.
- `apply_actions` : 승인 config에서는 shaping+IK를 physics rate에 수행한다. Position target write는 actual joint teleport가 아니다.
- `solve` : 이전 accepted target을 initial guess로, actual q를 neutral 후보로 사용한다. Warm-start shortcut이 있어 매 번 전체 seed를 푸는 것은 아니다.
- `reset_from_reference` : Placement-phase 선택 이후 IK reset을 동기화한다. 초기 joint write와 buffer 초기화는 reset에서 수행한다.
- `FrozenPolicyAdapter` 는 전용 비교 evaluator의 frozen 추론 검증 도구다. 모든 `rl.sh` 호출이 이 class를 반드시 거친다고 가정하지 않는다.

현재 video 경로는 1 env만 검증·지원한다. Floating 4096 env를 여기에 그대로 넣어도 arm training 처리량이 확보됐다고 할 수 없다.

## 20.4 변경별 검증 지도

| 바꾸려는 것                | 지켜야 할 계약                           | 기존 테스트 / 추가 실제 확인                                                                       |
|-----------------------|------------------------------------|-----------------------------------------------------------------------------------------|
| Hand model/keypoint   | Parent frame, leader order, mimic  | <code>test_revo2_kinematics.py</code> , Isaac FK                                        |
| 좌우/world 변환           | Reflection 대상, proper R, units     | <code>test_hand_mirroring.py</code> , <code>test_world_transform.py</code>              |
| Reference trim/loader | Quaternion, phase, dt, joint order | <code>test_reference_trajectory.py</code> , <code>test_rb3_revo2_rl_reference.py</code> |
| Mount/FK/IK           | Hand base↔link6 fixed transform    | <code>test_rb3_vertical_mount_usd.py</code> , <code>test_rb3_kinematics.py</code>       |

| 바꾸려는 것                 | 지켜야 할 계약                                       | 기존 테스트 / 추가 실제 확인                                                                      |
|------------------------|------------------------------------------------|----------------------------------------------------------------------------------------|
| Policy 입력/로딩           | 67/94/12, normalization, frozen weights        | test_frozen_policy_adapter.py , 동일 obs/action                                          |
| Response/rate/reset    | 30/120 Hz, stale history 없음                    | test_mounted_response.py , test_policy_rate_ik.py , timestamp                          |
| Gains/contact          | Baseline 격리, command/actual, torque provenance | test_arm_precision.py , test_revo2_contact_material.py , PhysX 비교                      |
| Singularity            | Strict/approx, speed/topology/collision        | test_velocity_bounded_ik.py , test_sequence_branch_ik.py , test_singularity_methods.py |
| Launcher/default model | Floating/transfer 혼선 없음                        | test_checkpoint_launchers.py , test_repository_entrypoints.py                          |
| 평가 방식                  | 동일 actual 초기 상태, terminal/reset 구분             | test_transfer_recovery.py , test_arm_transfer.py , closed-loop 비교                      |

테스트는 tests/에 있다. Unit test만으로 grasp를 증명하지 않는다. 반대로 물리 성능을 고친다면 quaternion·phase를 바꾸면 checkpoint 비교가 무효가 될 수 있다.

## 21. 용어·확인 문제·다음 공부

### 21.1 자주 혼동하는 말

| 용어                   | 여기서의 뜻                           | 아닌 것                          |
|----------------------|----------------------------------|-------------------------------|
| Retargeting          | 사람-물체 관계를 robot FK reference로 맞춤 | 물리 grasp 성공 학습                |
| Reference            | Residual의 기준 trajectory          | 매 step 강제하는 actual state      |
| Residual             | Policy의 12차원 보정                  | Motor torque / 추가 12 DoF      |
| Wrist                | right_hand_base_link SE(3)       | Link6 자체 / kp_00              |
| Leader / mimic       | 독립 6 joint / 종속 follower 관계      | 21점을 독립 제어                    |
| Phase                | Demo 진행 좌표                       | Wall-clock FPS                |
| Warm-start           | 이전 해로 최적화 초기화                    | 이전 frame을 복사해 성공 처리           |
| IK success           | Joint/pose tolerance 만족          | 추종·충돌·파지 성공                   |
| Kinematic replay     | 저장된 state를 보여 주기                 | 새 policy+physics 평가           |
| Zero agent           | Residual 0의 물리 actuator 실행       | Object reference teleport     |
| Deterministic policy | 같은 obs→같은 mean action            | 다른 physical state→같은 motion   |
| RSI                  | Reference 임의 frame reset         | 전체 episode 성공 보증              |
| Fine-tuning          | 같은 policy를 새 환경에서 계속 학습          | Frozen policy 뒤 새 policy 덧붙이기 |
| Real-time            | Wall time 대 simulation time      | 녹화 FPS만으로 보장                  |

### 21.2 확인 문제와 해설

| 질문                                          | 해설                                                                             |
|---------------------------------------------|--------------------------------------------------------------------------------|
| 1. 왜 action이 21×3차원이 아닌가?                   | 21점은 wrist와 leader 6의 FK 결과다. Policy는 wrist 6+leader 6 residual이다.             |
| 2. 왜 retargeter는 13 scalar인가?               | Unit quaternion을 4성분으로 저장한다. Physical DoF와 parameter 수가 다르다.                   |
| 3. Object만 10 cm 옮겨도 같은 동작인가?               | Hand·reference·canonical obs를 일관되게 옮겨야 한다. 이후에도 arm reachability/contact는 별도다. |
| 4. 왼손 변환은 180° 회전인가?                        | 아니다. Reflection determinant는 -1, proper rotation은 +1이다.                        |
| 5. Mesh min Z=0이면 바닥면이 수평인가?                | 기울어진 캔의 vertex 하나만 닿을 수 있다. Leveling은 별도 조건이다.                                 |
| 6. 30 Hz 명령이면 매 1/30초에 정확히 도착하나?            | 갱신 주기일 뿐이다. 유한 응답·관성·limits·shaping이 있다.                                       |
| 7. 2 mm 목표 변화에서 0.8 rad joint 변화는 반드시 버그인가? | Singularity/branch로 가능하다. Raw IK와 actual을 분리하고 Jacobian·다른 branch를 검사한다.       |
| 8. C 오차 1 mm면 floating과 같은 파지인가?            | Shaping 전후 차이, 손가락, 속도, contact timing/force가 남을 수 있다.                         |
| 9. 20/20 success면 일반 성공률 100%인가?            | 해당 배치 관측이며 success 정의도 확인한다. Held-out과 lift/hold/drop이 별도다.                    |

| 질문                                         | 해설                                                          |
|--------------------------------------------|-------------------------------------------------------------|
| 10. +60° offline IK 통과는 policy grasp 성공인가? | 아니다. 목표/branch가 다르고 yaw canonicalization·contact 검증이 남아 있다. |
| 11. model_9999.pt 는 영상 9999개로 학습했나?        | Run의 update 번호다. Env×rollout transitions와 budget을 확인한다.     |
| 12. 더 오래 학습하면 frame 오류도 해결되나?              | 데이터/관측/frame 오류를 학습으로 덮지 말고 계약부터 맞춰야 한다.                    |

계산 연습:

- 30 Hz translation residual 한 축 0.3 → **0.01 m** 보정. Reference에 더하는 위치 보정이며 매 step 1 cm씩 누적하지 않는다.
- Finger residual 0.5 → **0.053333 rad ≈ 3.056°**. Native limit를 넘으면 후단에서 제한한다.
- 120 Hz raw step 0.8 rad → 차분 **96 rad/s**. 10 rad/s bound의 한 tick target step은 **0.083333 rad**다. Actual 속도가 자동으로 이 값과 같지는 않다.
- 38 frame, 30 Hz의 첫-마지막 sample 간격 → **37/30 ≈ 1.233 s**. 영상 end-hold·배속과 phase/dt는 별개다.

### 21.3 다 읽은 뒤 설명할 수 있어야 하는 것

1. 입력 frame·point order와 mirror 대상이 무엇인지 설명한다.
2. Local keypoint→leader/mimic FK→Interaction Mesh objective를 식으로 연결한다.
3. Reference가 observation·decoder·reward·RSI에서 어떻게 쓰이는지 구분한다.
4. Mounted actual→policy→wrist target→IK→command→physics의 두 시간축을 설명한다.
5. Offline FK 가능성·actual tracking·grasp evidence를 구별한다.

| 더 궁금한 것             | 다음 자료                        |
|---------------------|------------------------------|
| 데이터 생성/convention   | DATA_PIPELINE.md             |
| 학습/평가/checkpoint    | RL_TASK.md                   |
| Isaac replay/marker | ISAAC_SIM_REPLAY.md          |
| 팔 제어 수정 이유          | ARM_CONTROL_HISTORY_KO.md    |
| Singularity 후보 결과   | ARM_IK_SINGULARITY_FIX.md    |
| 책상 가능 위치·방향         | TABLETOP_OPERATING_REGION.md |
| 실행/검증               | scripts/README.md, tests/    |

이 프로젝트는 물리 파지 **policy** 구조를 보존하며 다른 손·팔·데이터·시뮬레이터 경계에 연결하는 작업이다. 리타게팅 식 하나, IK solver 하나, gain 숫자 하나가 전체 성공을 보장하지 않는다. 재현 가능한 입력과 실제 상태를 바탕으로 각 경계의 의미를 확인하는 것이 가장 중요한 공부 방법이다.